Group: Sara Martinez Rendon (1000424179) | Heider Zapata (1000009586) | Yeison Londoño (1000009479)
Master's Degree in Data Sciences | EAFIT University
Course: Applied Mathematics
Date: 24/02/2026

Examen 2 - Matemáticas Aplicadas en Ciencia de Datos

Ejercicio 1

Siguiendo el paso a paso planteado en el enunciado del examen, sin saber como se distribuyen los datos en un gráfico:

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import KMeans
from scipy.linalg import eigh

#Ruta dentro del repositorio
ruta = "data/raw/datos_ejercicio1.csv"

df = pd.read_csv(ruta)
print(df.head())

print("")
print("Tamaño de los datos:")
print(df.shape)

print("")
n = df.shape[0]
d = df.shape[1]
print(f"Numero de observaciones (n): {n}")
print(f"Dimension de las observaciones (d): {d}")
```

```
   componente_1  componente_2
0     -4.168461      2.893627
1      3.550935      3.114696
2      4.926501     -0.184713
3      3.823268      3.409679
4      1.795431     -4.607159

Tamaño de los datos:
(1950, 2)

Numero de observaciones (n): 1950
Dimension de las observaciones (d): 2
```

1.1. Matriz de similitud

Este ejercicio tiene que ver con los grafos en su interior porque son puntos (nodos) y hallando al Matriz de Similitud (S) se halla un tipo de matriz de adyacencia, solo que en lugar de 0 o , probabilidades como en Markiv-Chain, se usa un número constante que se calcula con la formula para los vecinos conectados, como se propone en el enunciado:

$$S_{ij} = \frac{1}{\|x_i - x_j\| + \varepsilon}$$

Sabiendo que:

- A **menor distancia** → **mayor similitud** (son más parecidos).
- ε = 0.00001 es un número diminuto que evita dividir entre cero si dos puntos coinciden.

Sabiendo que $\|x_i - x_j\|$ es la distancia entre los puntos, procedemos a calcular la matriz S.

Usando KNN = 15 para saber la distancia entre cada punto y sus 15 vecinos más cercanos.

In [24]:
```
#DEfiniendo variables
kNN = 15
epsilon = 0.00001
df_array = df.values


#Distancia entre cada punto y sus kNN = 15 y el mismo, por eso se pone kNN + 1
#NearestNeighbors devuelve dos matrices: distances (distancia a cada vecino) e indi
  #y aace el calculo de la distancia euclidiana por defecto.
#'ball_tree' es un algoritmo eficiente para encontrar vecinos cercanos en espacios
nbrs = NearestNeighbors(n_neighbors=kNN + 1, algorithm='ball_tree').fit(df_array)
distances, indices = nbrs.kneighbors(df_array)

# Inicializamos S con ceros
S = np.zeros((n, n))

for i in range(n):
    for j_idx in range(1, kNN + 1):
        j = indices[i, j_idx]
        dist = distances[i, j_idx]
        S[i, j] = 1.0 / (dist + epsilon)
```

```python
print(S[:5, :5])  # Imprime una parte de la matriz S para verificar

#Eliminando el problema practico de la diagonal
S = np.maximum(S, S.T) #esta función compara S con su transpuesta y se queda con el


print("\n Matriz S simetrizada:")
print(S[:5, :5])  # Imprime una parte de la matriz S para verificar

#print("\nNúmero de vecinos para cada punto (debería ser 15):")
#for i in range(n):
    #print(f"Punto {i}: {np.sum(S[i, :] > 0)} vecinos")

print("Verificando la matriz S:")
print("Dimensiones de S:", S.shape)
print("Valor mínimo de S:", np.min(S))
print("Valor máximo de S:", np.max(S))

#Verificando que el mismo número no quede entre sus vecinos
print("\nVerificando que el mismo número no quede entre sus vecinos:")
print(f"Indices de los {kNN} vecinos de la observacion 0:")
print(indices[0, 1:])   # desde 1 para saltar el propio punto

print(f"\n Distancias a esos vecinos:")
print(distances[0, 1:].round(4))

print(f"\nValor de S para esos vecinos:")
print(" ".join([f"{S[0, j]:.4f}" for j in indices[0, 1:]]))
```

```
[[0.         0.         0.         0.         0.        ]
 [0.         0.         0.         0.         0.        ]
 [0.         0.         0.         0.         0.        ]
 [0.         2.49076826 0.         0.         0.        ]
 [0.         0.         0.         0.         0.        ]]

 Matriz S simetrizada:
[[0.         0.         0.         0.         0.        ]
 [0.         0.         0.         2.49076826 0.        ]
 [0.         0.         0.         0.         0.        ]
 [0.         2.49076826 0.         0.         0.        ]
 [0.         0.         0.         0.         0.        ]]
Verificando la matriz S:
Dimensiones de S: (1950, 1950)
Valor mínimo de S: 0.0
Valor máximo de S: 414.04837987233475

Verificando que el mismo número no quede entre sus vecinos:
Indices de los 15 vecinos de la observacion 0:
[633 524 238 201 788 279 564 291 572 148 208 377 371 687 661]

 Distancias a esos vecinos:
[0.0316 0.0803 0.0822 0.0876 0.1106 0.1173 0.1232 0.129  0.1599 0.1622
 0.1656 0.1826 0.192  0.2058 0.2112]

Valor de S para esos vecinos:
31.5972 12.4551 12.1637 11.4137 9.0436 8.5275 8.1137 7.7530 6.2534 6.1630 6.0395 5.4
761 5.2079 4.8591 4.7346
```

1.2.Matriz Laplaciana

- Matriz D: en término generales recordar que es una matriz diagonal con el número de conexiones de cada punto, que aquí sería el número de similitudes con sus vecinos.

In [25]:
```
vector_D = S.sum(axis=1)          # vector de n números
D = np.diag(vector_D)             # convertir a matriz diagonal n×n

print(D[:5,:5]) #Ver las primeras 5 diagonales de la matriz D
```

```
[[157.79486273   0.           0.           0.           0.        ]
 [  0.          80.05076245   0.           0.           0.        ]
 [  0.           0.          98.21308397   0.           0.        ]
 [  0.           0.           0.          74.12940381   0.        ]
 [  0.           0.           0.           0.          91.39638687]]
```

Ahora sí se calcula la matriz Laplaciana, que recordando el ejemplo de clase con redes sociales, nos permite *revelar comunidades en las redes sociales*, y matemáticamente se describe como:

$$L = D - S$$

In [31]:
```
L = D - S

print("Verificación de la matriz L:")
```

```
print(f"    L es simétrica: {np.allclose(L, L.T)}")
print(L[:5,:5]) #Ver las primeras 5 diagonales de la matriz D
```

```
Verificación de la matriz L:
    L es simétrica: True
[[157.79486273   0.           0.           0.           0.        ]
 [  0.          80.05076245   0.          -2.49076826   0.        ]
 [  0.           0.          98.21308397   0.           0.        ]
 [  0.          -2.49076826   0.          74.12940381   0.        ]
 [  0.           0.           0.           0.          91.39638687]]
```

1.3. Análisis de Eigenpares

Para efectos de esta actividad la matriz Laplaciana (L), es nuestra matriz de transformación lineal. Es decir, si le aplicas L a ciertos vectores especiales (los **eigenvectores**), el vector no cambia de dirección, solo se estira o encoge. El factor por el que se estira es el **eigenvalor**:

$$L \cdot v = \lambda \cdot v$$

donde v es el eigenvector y λ es el eigenvalor.

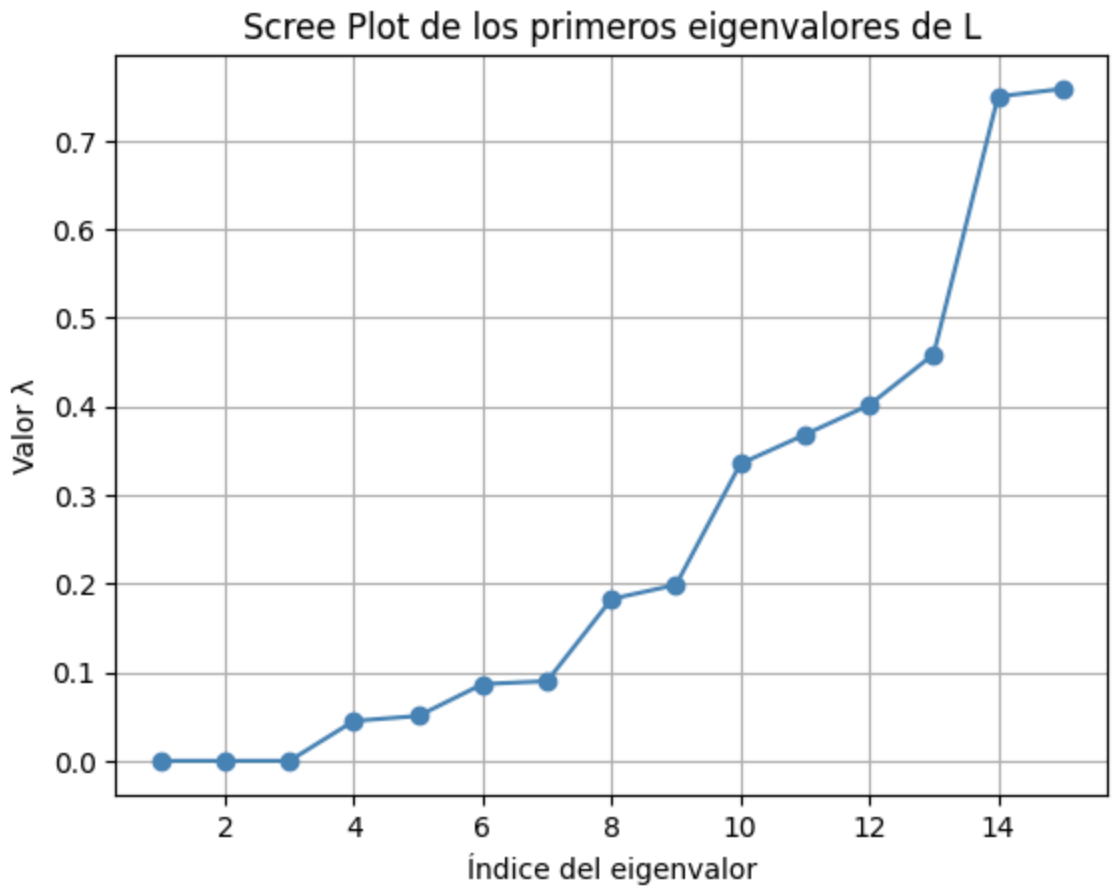
Nota: Debemos tomar en cuenta que el número de eigenvalores iguales a cero (0) de L representa al número de componentes conexas (aislados) del grafo.

In [36]:
```
print("\nCalculando los valores y vectores propios de L...")
eigenvalues, eigenvectors = np.linalg.eigh(L) #eigh es para matrices simétricas, de

print(f"   Total de eigenvalores: {len(eigenvalues)}")

print(f"Los primeros 10 valores propios ordenados: {eigenvalues[:15]}")


print(f"\n--- Primeros 15 eigenvalores ---")
for i, lam in enumerate(eigenvalues[:15]):
    indicador = " ← ≈ 0  (aislamiento)" if abs(lam) < 0.001 else ""
    print(f"   λ_{i+1:2d} = {lam:10.6f}{indicador}")
```

```
Calculando los valores y vectores propios de L...
   Total de eigenvalores: 1950
Los primeros 10 valores propios ordenados: [-4.78842077e-14 -5.35444102e-15  2.55195
651e-14  4.50512996e-02
  5.06502842e-02  8.66981683e-02  8.99720446e-02  1.82728201e-01
  1.98607008e-01  3.35238783e-01  3.68165043e-01  4.01573960e-01
  4.58899501e-01  7.49848480e-01  7.58492625e-01]

--- Primeros 15 eigenvalores ---
   λ_ 1 =  -0.000000 ← ≈ 0  (aislamiento)
   λ_ 2 =  -0.000000 ← ≈ 0  (aislamiento)
   λ_ 3 =   0.000000 ← ≈ 0  (aislamiento)
   λ_ 4 =   0.045051
   λ_ 5 =   0.050650
   λ_ 6 =   0.086698
   λ_ 7 =   0.089972
   λ_ 8 =   0.182728
   λ_ 9 =   0.198607
   λ_10 =   0.335239
   λ_11 =   0.368165
   λ_12 =   0.401574
   λ_13 =   0.458900
   λ_14 =   0.749848
   λ_15 =   0.758493
```

Scree plot para visualizar los eigenvalores e identificar los saltos entre los eigenvalores:

In [39]:
```python
n_show = 15

plt.plot(range(1, n_show+1), eigenvalues[:n_show], 'o-', color='steelblue')


plt.xlabel('Índice del eigenvalor')
plt.ylabel('Valor λ')
plt.title('Scree Plot de los primeros eigenvalores de L')
plt.grid(True)
plt.show()
```



In [40]:
```
# ── Seleccionar k ──────────────────────────────────────────────────────
k = 3   # Tres eigenvalores ≈ 0 → tres componentes conexas → k = 3
print(f"\n🎯 k seleccionado: k = {k}")
print(f"   Razón: {k} eigenvalores son prácticamente 0")
print(f"   ({', '.join([f'λ_{i+1}={eigenvalues[i]:.2e}' for i in range(k)])})")
print(f"   Seguido de un salto a λ_{k+1} = {eigenvalues[k]:.4f}")
```

```
🎯 k seleccionado: k = 3
   Razón: 3 eigenvalores son prácticamente 0
   (λ_1=-4.79e-14, λ_2=-5.35e-15, λ_3=2.55e-14)
   Seguido de un salto a λ_4 = 0.0451
```

Del plot y de los valores propios se observa que los 3 primeros son practivamente cero, y que entre el tercer (λ_3) y el cuarto (λ_4) pasa de ≈0 a 0.045051; lo que indica una brecha y ese eigenvalor nos indica que **k = 3**

1.4. Construcción del Espacio de Embedding

Definiendo el espacio embbeding espectral:

- Espacio: cada punto está en un espacio R^2, dado que solo hay dos variables (componente_1, componente_2).
- Embbeding: es re-expresar cada punto en un nuevo sistema de coordenadas, tal como se hace con PCA.

Entonces, ¿qué ahce el embbeding espectral? en lugar de usar los eigenvectores (direcciones de mayor varianza) de la matriz de covarianca (PCA), se usan los eigenvectores de la Laplaciana (L).

In [ ]:
```
#Sabiendo que el k = 3
#Cada columna es el punto (vector R_2) multiplicado por el eigenvector correspondie
#Al hacer la multiplicación se obtiene la nueva representación del punto i en el es
#conexas = aislados

Y = eigenvectors[:, :k]   # shape: (n, k) = (1950, 3)

print(f"Embedding espectral Y:")
print(f"   Shape: {Y.shape}  ← (n_puntos, k_dimensiones)")
print(f"   Cada fila yᵢ = [ v₁(i), v₂(i), v₃(i) ] es la nueva representación del pu
print(f"\nPrimeras 5 filas de Y (embedding de los primeros 5 puntos):")
print(pd.DataFrame(Y[:5], columns=['v₁', 'v₂', 'v₃']).round(7).to_string())
```

```
Embedding espectral Y:
   Shape: (1950, 3)  ← (n_puntos, k_dimensiones)
   Cada fila yᵢ = [ v₁(i), v₂(i), v₃(i) ] es la nueva representación del punto i

Primeras 5 filas de Y (embedding de los primeros 5 puntos):
    v₁        v₂   v₃
0  0.0  0.035355  0.0
1  0.0  0.035355  0.0
2  0.0  0.035355  0.0
3  0.0  0.035355  0.0
4  0.0  0.035355  0.0
```

1.5. Agrupamiento

Siguiendo la recomendación del enunciado se usará como método de agrupación el K-Means.

¿Qué sabemos de K-MEans? Es un método clásico de clustering/agrupación, y su algoritmo consiste:

1. Coloca k centroides al azar en el espacio.
2. Asigna cada punto al centroide más cercano.
3. Recalcula cada centroide como el **promedio** de sus puntos asignados.
4. Repite 2 y 3 hasta que nada cambia.

K-means funciona bien cuando los grupos son **bolitas compactas y separadas** en el espacio donde se aplica. En el espacio de embedding Y, eso es exactamente lo que tenemos.

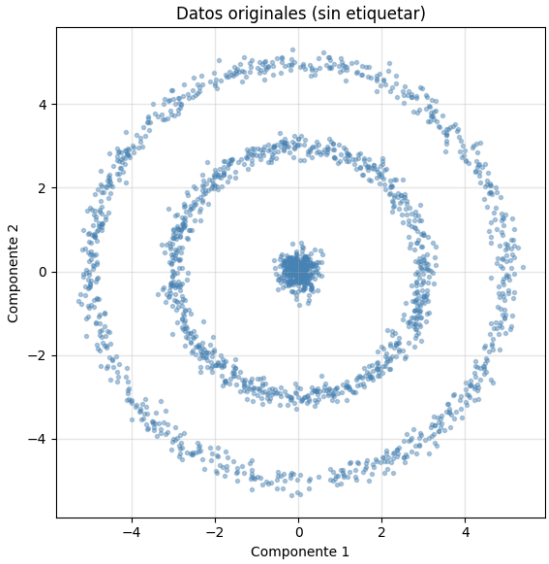
In [ ]:
```
#K-Means sobre el embedding espectral Y
kmeans_spectral = KMeans(
    n_clusters=k,       # k=3
    n_init=30,          # 30 inicializaciones distintas → para tomar la mejor
    random_state=42     # semilla para reproducibilidad, 42 por convención pópular
)
labels_spectral = kmeans_spectral.fit_predict(Y)  # etiquetas para cada punto
```

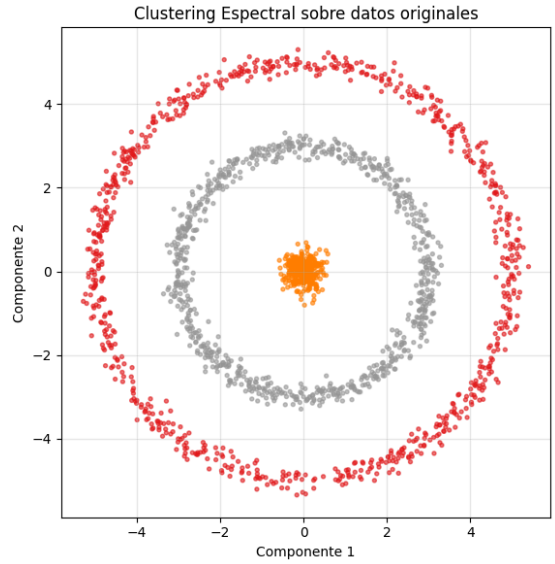
```
#Ahora verificando que los cluster di hiciesen la agrupación correcta, es decir, qu
# en el mismo cluster tengan un radio similar (distancia al origen) y que los clust
print(f"\nResultados del Clustering Espectral:")
for c in range(k):
    mask = labels_spectral == c
    r_c = np.sqrt(df_array[mask,0]**2 + df_array[mask,1]**2)
    print(f"   Cluster {c}: {mask.sum():4d} puntos | radio medio = {r_c.mean():.2f}

print(f"\n   Inercia (suma de distancias² al centroide): {kmeans_spectral.inertia_:
print(f"Tipo de labels_spectral: {type(labels_spectral)}, su forma: {labels_spectra
```

```
Resultados del Clustering Espectral:
   Cluster 0:  800 puntos | radio medio = 4.99 ± 0.15
   Cluster 1:  350 puntos | radio medio = 0.29 ± 0.15
   Cluster 2:  800 puntos | radio medio = 3.00 ± 0.15

   Inercia (suma de distancias² al centroide): 0.0000
Tipo de labels_spectral: <class 'numpy.ndarray'>, su forma: (1950,), valores únicos:
[0 1 2]
```

In [50]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Panel izquierdo: datos originales sin etiquetar
axes[0].scatter(df_array[:, 0], df_array[:, 1], c='steelblue', s=8, alpha=0.4)
axes[0].set_title('Datos originales (sin etiquetar)')
axes[0].set_xlabel('Componente 1')
axes[0].set_ylabel('Componente 2')
axes[0].set_aspect('equal')
axes[0].grid(True, alpha=0.3)

# Panel derecho: datos originales coloreados por cluster espectral
axes[1].scatter(df_array[:, 0], df_array[:, 1], c=labels_spectral, cmap='Set1', s=8
axes[1].set_title('Clustering Espectral sobre datos originales')
axes[1].set_xlabel('Componente 1')
axes[1].set_ylabel('Componente 2')
axes[1].set_aspect('equal')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```





1.6. a. Implementación de K-Means al espacio original

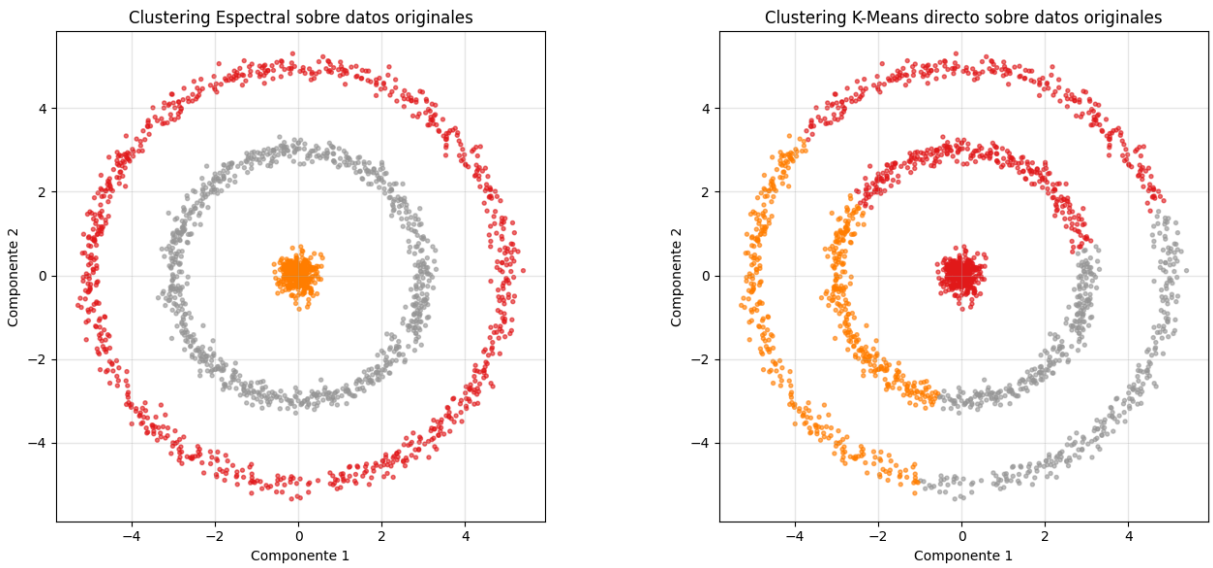
Para comparar el K-Means aplicado al espacio espectral y K-Means a los datos originales, aplicamos K-Means a los datos originales:

In [47]:
```
#K-Means sobre los puntos originales X (sin usar el embedding espectral)
kmeans_original = KMeans(
    n_clusters=k,       # k=3
    n_init=30,          # 30 inicializaciones distintas → para tomar la mejor
    random_state=42     # semilla para reproducibilidad, 42 por convención pópular
)
labels_original = kmeans_original.fit_predict(df_array)  # etiquetas para cada punt


#Ahora verificando que los cluster di hiciesen la agrupación correcta, es decir, qu
# en el mismo cluster tengan un radio similar (distancia al origen) y que los clust
print(f"\nResultados del Clustering Original:")
for c in range(k):
    mask = labels_original == c
    r_c = np.sqrt(df_array[mask,0]**2 + df_array[mask,1]**2)
    print(f"   Cluster {c}: {mask.sum():4d} puntos | radio medio = {r_c.mean():.2f}

print(f"\n   Inercia (suma de distancias² al centroide): {kmeans_original.inertia_:
print(f"Tipo de labels_original: {type(labels_original)}, su forma: {labels_origina
```

```
Resultados del Clustering Original:
   Cluster 0:  882 puntos | radio medio = 2.50 ± 1.96
   Cluster 1:  555 puntos | radio medio = 4.02 ± 1.01
   Cluster 2:  513 puntos | radio medio = 4.01 ± 1.01

   Inercia (suma de distancias² al centroide): 11814.0260
Tipo de labels_original: <class 'numpy.ndarray'>, su forma: (1950,), valores únicos:
[0 1 2]
```

Ahora gráficamos el clustering con el espacio original y el clustering espectral:

In [52]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Panel derecho: datos originales coloreados por cluster espectral
axes[0].scatter(df_array[:, 0], df_array[:, 1], c=labels_spectral, cmap='Set1', s=8
axes[0].set_title('Clustering Espectral sobre datos originales')
axes[0].set_xlabel('Componente 1')
axes[0].set_ylabel('Componente 2')
axes[0].set_aspect('equal')
axes[0].grid(True, alpha=0.3)

# Panel derecho: datos originales coloreados por agrupoación K-Means sobre datos or
axes[1].scatter(df_array[:, 0], df_array[:, 1], c=labels_original, cmap='Set1', s=8
axes[1].set_title('Clustering K-Means directo sobre datos originales')
axes[1].set_xlabel('Componente 1')
axes[1].set_ylabel('Componente 2')
axes[1].set_aspect('equal')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



CONCLUSIONES:

- El clustering embbeding espectral propuesto para aprender en este ejercicio, pertmire separar bien los grupos con forma de anillo al considerarse la silimitud. Mientras que el clustering directo usa distancias al centroide y crea fronteras rectas, despreciando la forma circular de los datos.
- K-Means lineal siempre espera que los datos estén agrupados en pequeños grupitos ("bolitas compactas"), basandose en las coordenada brutas de los puntos, y siguiendo una distancia lineal entre el centroide y el punto.
- El clustering espectral se basa en la similaridad de un punto a sus vecinos, es decir con la estructura de conexiones del grafo, lo que hace que los clusters tomen formas de anillos.
- Los eigenvalores de la Laplaciana nos permite saber cuántos grupos hay.

- El número de kNN importa porque si es muy pequeño, algunos puntos pueden quedar desconectados dentro de su propio anillo.

1.6.b. Cambiando funciones de similitud

Para elegir una función de similitud válida se deben cumplir dos condiciones básicas:

1. A menor distancia → mayor similitud.
2. A mayor distancia → similitud cercana a cero.

Se evaluarán tres funciones y se compararán sus resultados:

Función 1 — Inversa de distancia (original del ejercicio):

$$S_{ij} = \frac{1}{\|x_i - x_j\| + \varepsilon}$$

Cumple las dos condiciones pero crece sin límite cuando los puntos son muy cercanos, lo que puede hacer que unos pocos pares de puntos dominen toda la matriz S.

Función 2 — Gaussiana:

$$S_{ij} = e^{-\frac{\|x_i - x_j\|^2}{2\sigma^2}}$$

También cumple las dos condiciones, con la ventaja de que sus valores siempre están acotados entre 0 y 1. El parámetro σ controla qué tan rápido decae la similitud con la distancia:

- σ pequeño → solo los vecinos muy cercanos tienen similitud significativa.
- σ grande → vecinos más lejanos también reciben similitud alta.

Función 3 — Inversa escalada por la distancia promedio:

$$S_{ij} = \frac{\bar{d}}{\|x_i - x_j\| + \varepsilon}$$

Donde $\bar{d}$ es la distancia promedio entre todos los pares de vecinos del grafo. Es una variación de la función original que usa el promedio de distancias como referencia de escala. En la práctica produce valores de similitud comparables entre sí, evitando que pares de puntos muy cercanos dominen la matriz S.

In [53]:
```
#Pipeline reutilizable para clustering espectral, con cualquier función que se ejec
def clustering_espectral(X, indices, distances, kNN, sim_func, k, epsilon):
    n = X.shape[0]

    # Construir S
    S = np.zeros((n, n))
    for i in range(n):
        for j_idx in range(1, kNN + 1):
```

```
            j = indices[i, j_idx]
            dist = distances[i, j_idx]
            S[i, j] = sim_func(dist)
    S = np.maximum(S, S.T)

    # Construir L
    D = np.diag(S.sum(axis=1))
    L = D - S

    # Eigenpares
    eigenvalues, eigenvectors = eigh(L)

    # Embedding y K-Means
    Y = eigenvectors[:, :k]
    labels = KMeans(n_clusters=k, n_init=30, random_state=42).fit_predict(Y)

    return labels, eigenvalues
```

In [58]:

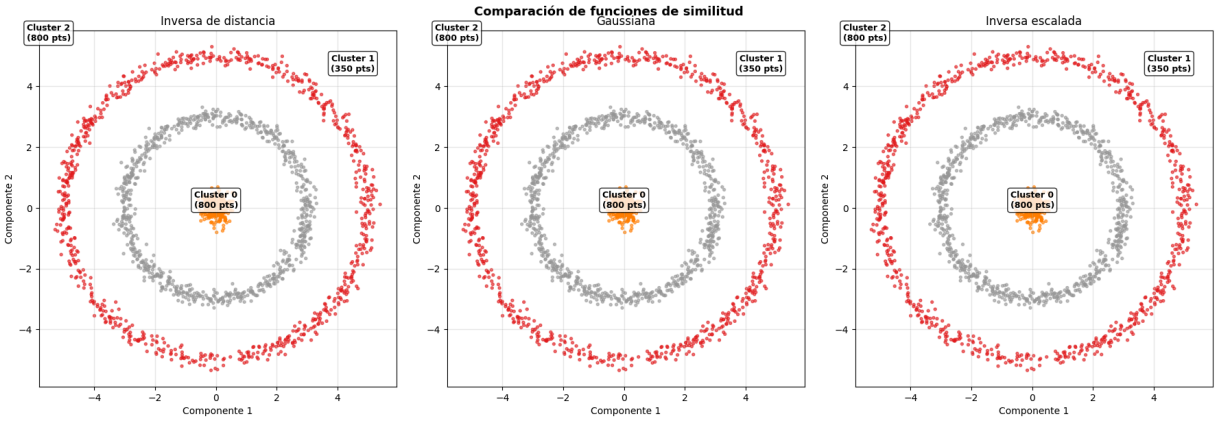
```
#Funciones y parametros necesarios para ejecutar el pipeline

sigma = 1.5  # parámetro de la Gaussiana

#parámetro para calcular la distancia promedio de la matriz de distancias.
d_promedio = distances[:, 1:].mean()  # ignoramos columna 0 (distancia al propio pu
print(f"Distancia promedio entre vecinos: {d_promedio:.4f}")

funciones_similitud = {
    'Inversa de distancia (original)': lambda d: 1.0 / (d + epsilon),
    'Gaussiana':            lambda d: np.exp(-d**2 / (2 * sigma**2)),
    'Inversa escalada (d_promedio / (d + epsilon)':     lambda d: d_promedio / (d +
}
```

```
Distancia promedio entre vecinos: 0.1887
```

In [57]:

```
#Visualizaciones

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

posiciones = {0: (0, 0), 1: (4.5, 4.5), 2: (-5.5, 5.5)}

for ax, (nombre, resultado) in zip(axes, resultados.items()):
    labels = resultado['labels']

    ax.scatter(df_array[:, 0], df_array[:, 1], c=labels, cmap='Set1', s=8, alpha=0.

    for c in range(k):
        mask = labels == c
        cx, cy = posiciones[c]
        ax.annotate(f'Cluster {c}\n({mask.sum()} pts)', (cx, cy),
                    fontsize=9, fontweight='bold', ha='center',
                    bbox=dict(boxstyle='round', fc='white', alpha=0.8))

    ax.set_title(nombre)
    ax.set_xlabel('Componente 1')
    ax.set_ylabel('Componente 2')
    ax.set_aspect('equal')
```

```
    ax.grid(True, alpha=0.3)

plt.suptitle('Comparación de funciones de similitud', fontsize=13, fontweight='bold
plt.tight_layout()
plt.show()
```



De los scatterplots para las 3 funciones de similitud son "idénticas", sin embargo, esto puede no deberse completamente a las funciones de similitud propuestas, sino que también influye que los datos tienen una estructura clara y bien definida, además que cada anillos están bien separados.

Ejercicio 2

> - **En esta actividad trabajarás con imágenes de dígitos escritos a mano utilizando el conjunto de datos MNIST (disponible en Keras). Tu tarea es implementar un clasificador simple basado en la utilización del conceptos de álgebra lineal.**

Como primer paso, comprendamos el tipo de datos que vamos a manipular:

¿Qué es MNIST?

MNIST es una base de datos que contiene 70,000 imágenes pequeñas (de 28x28 píxeles). Cada imagen es un dígito del 0 al 9 escrito a mano por diferentes personas. Las imagenes están procesadas en una estructura de tensores (matrices multidimensionales):

Objeto	Rol	Forma (Shape)	Rango de Valores	Descripción
`x_train`	Entrenamiento	`(60000, 28, 28)`	`0 - 255`	**Tensor 3D:** Un "bloque" de 60,000 matrices. Cada matriz representa una imagen (digito/numero dibujado) y está compuesta por 28×28 píxeles. Cada

Objeto	Rol	Forma (Shape)	Rango de Valores	Descripción
				pixel indica el color o intensidad del lapiz (0: ausencia, 255: maxima presencia del trazo).
`y_train`	Entrenamiento	`(60000,)`	`0 - 9`	**Tensor 1D (Vector)** Contiene la etiqueta real (el número dibujado) para cada imagen del set de entrenamiento. La relación entre las imágenes dibujadas y sus etiquetas es implícita por su posición (índice).
`x_test`	Evaluación	`(10000, 28, 28)`	`0 - 255`	**Tensor 3D**: 10,000 imágenes adicionales que usaremos para validar la capacidad de generalización de nuestro clasificador
`y_test`	Evaluación	`(10000,)`	`0 - 9`	**Tensor 1D (Vector)**: Las etiquetas reales del set de prueba para medir el nivel de precisión del clasificador.

Al ejecutar `(x_train, y_train), (x_test, y_test) = mnist.load_data()`, el módulo de TensorFlow descarga y organiza automáticamente los datos en estos cuatro objetos de tipo **NumPy Array**.

Fuente: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import cv2 # Usaremos OpenCV para el reescalado
from tensorflow.keras.datasets import mnist

# 1. Cargar el conjunto de datos MNIST
(X_train_full, y_train_full), (X_test_full, y_test_full) = mnist.load_data()

# Veriquemos dimensiones
print(f"X_train_full.shape: {X_train_full.shape}")
print(f"y_train_full.shape: {y_train_full.shape}")
print(f"X_test_full.shape: {X_test_full.shape}")
print(f"y_test_full.shape: {y_test_full.shape}")


# Veamos la matriz que representa la primera imagen (primer dígito) de X_train_full
print("")
print("Primer digito en la matriz de entrenamiento (X_train_full[0]):")
#X_train_full[0]
  # Usamos un for para que nos muestre cada número de la matriz (pixel) en un forma
  # poder ver la matriz completa en un formato 28*28
for fila in X_train_full[0]:
    # Imprime cada número con 3 espacios de ancho para mantener la cuadrícula
    contenido_fila = "".join(f"{pixel:3}" for pixel in fila)
    print(contenido_fila)
```

```python
#Veamos la primer etiqueta de y_train_full
print("")
print("Etiqueta de y_train_full[0]:", y_train_full[0])
```

```
X_train_full.shape: (60000, 28, 28)
y_train_full.shape: (60000,)
X_test_full.shape: (10000, 28, 28)
y_test_full.shape: (10000,)

Primer digito en la matriz de entrenamiento (X_train_full[0]):
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  3 18 18 18126136175 26166255247127  0  0  0  0
  0  0  0  0  0  0  0  0 30 36 94154170253253253253253225172253242195 64  0  0  0  0
  0  0  0  0  0  0  0 49238253253253253253253253253251 93 82 82 56 39  0  0  0  0  0
  0  0  0  0  0  0  0 18219253253253253253198182247241  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0 80156107253253205 11  0 43154  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0 14  1154253 90  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0139253190  2  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0 11190253 70  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0 35241225160108  1  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0 81240253253119 25  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0 45186253253150 27  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 16 93252253187  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0249253249 64  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0 46130183253253207  2  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0 39148229253253253250182  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0 24114221253253253253201 78  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0 23 66213253253253253198 81  2  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0 18171219253253253253195 80  9  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0 55172226253253253253244133 11  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0136253253253212135132 16  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0

Etiqueta de y_train_full[0]: 5
```

2.1. Selección y Preprocesamiento de Datos:

> - **Para cada dígito $d = 0, 1, \ldots, 9$, selecciona aleatoriamente k imágenes de entrenamiento. Esto te dará un conjunto de datos de tamaño $10 \times k$.**

In [2]:
```python
# Definimos el parámetro k (número de imágenes por dígito) de manera aleatoria.
# Fijamos una semilla aleatoria para que el resultado sea reproducible
np.random.seed(42)
k = np.random.randint(100, 2000) # limites razonables
print(f"El valor de k generado aleatoriamente para esta ejecución es: {k}")
```

```
# Diccionario para almacenar las imágenes crudas (con 28x28 pixeles) seleccionadas
imagenes_crudas_k = {}



for d in range(10):
    # 1. Encontrar todos los índices en y_train_full que corresponden al dígito 'd'
    indices_d = np.where(y_train_full == d)[0]

    # 2. Seleccionar aleatoriamente 'k' índices de esa lista, sin reemplazo
    indices_seleccionados = np.random.choice(indices_d, k, replace=False)

    # 3. Extraer las matrices (digitos de 28x28) correspondientes a esos índices
    # X_train_full[indices_seleccionados] nos devuelve un arreglo de forma (k, 28,
    imagenes_crudas_k[d] = X_train_full[indices_seleccionados]

print(f"Diccionario creado. Contiene {len(imagenes_crudas_k)} llaves (una por dígit
print(f"Dimensiones de los datos guardados para el dígito 0: {imagenes_crudas_k[0].
```

```
El valor de k generado aleatoriamente para esta ejecución es: 1226
Diccionario creado. Contiene 10 llaves (una por dígito).
Dimensiones de los datos guardados para el dígito 0: (1226, 28, 28)
```

Confirmamos que para cada digito tenemos:

- k imágenes.
- Cada matriz tiene 28 filas.
- Cada matriz tiene 28 columnas.

In [3]:
```
# --- Visualización del resultado del muestreo ---
# Mostraremos un máximo de 5 imágenes por dígito para no saturar la pantalla,
# independientemente de qué tan grande sea 'k'.
num_mostrar = min(k, 5)

fig, axes = plt.subplots(10, num_mostrar, figsize=(num_mostrar * 1.5, 12))

for d in range(10):
    for i in range(num_mostrar):
        # Seleccionamos el eje correspondiente
        ax = axes[d, i]
        # Mostramos la imagen cruda (matriz 28x28)
        ax.imshow(imagenes_crudas_k[d][i], cmap='gray')
        ax.axis('off')

        # Ponemos el título solo en la primera columna para no saturar
        if i == 0:
            ax.set_title(f"Dígito {d}", loc='left')

plt.tight_layout()
plt.show()
```

Dígito 0

Dígito 1

Dígito 2

Dígito 3

Dígito 4

Dígito 5

Dígito 6

Dígito 7

Dígito 8

Vemos que tenemos un conjunto de datos de $10 \times k$ => 10 digitos con los primeros 5 elementos cada uno.

Notése que no forzamos un único tensor global de dimensiones (10, k, 28, 28) ya que, al guardar las matrices de cada digito en un diccionario, ya tenemos los datos pre-agrupados por clase (dígito), lo cual nos servirá para luego calcular la base ortonormal B_d de manera independiente para cada dígito.

> - **Convierte cada imagen a una matriz. Luego, recorta la imagen para eliminar las filas y columnas con únicamente el fondo (de modo que se preserve la mayor parte del dígito). Luego reescala la imagen resultante a un tamaño fijo de 16×16 píxeles.**

Dado que las imágenes importadas ya se encuentran en formato matricial (arreglos de NumPy de 28x28) gracias a la librería de Keras (`mnist.load_data()`), entonces procedemos a eliminar la información redundante:

- **Recorte (Bounding Box)**: Eliminamos los bordes de fondo negro (píxeles con valor 0). En álgebra lineal, estas áreas representan dimensiones con varianza nula que no aportan información para la clasificación. Para aislar la "señal" del "ruido", colapsamos la matriz bidimensional calculando el valor máximo a lo largo de sus ejes (filas y columnas). Esto nos permite identificar los índices exactos donde la intensidad del píxel deja de ser 0 . Al recortar la matriz usando estos índices extremos (la caja delimitadora o bounding box), conservamos únicamente la región espacial que contiene la estructura topológica del dígito.

- **Reescalado**: Como cada dígito recortado tendrá un tamaño diferente (por ejemplo, un "1" suele ser más angosto que un "8"), estandarizamos todas las matrices resultantes a un tamaño espacial fijo de 16×16 píxeles utilizando la interpolación de OpenCV. Este paso es crucial porque, para poder operar algebraicamente con ellas (ej. construir las matrices M_d y calcular la SVD), es estrictamente necesario que todas habiten en la misma dimensión geométrica.

Nota: ¿Por qué reescalar con cv2.INTER_AREA?

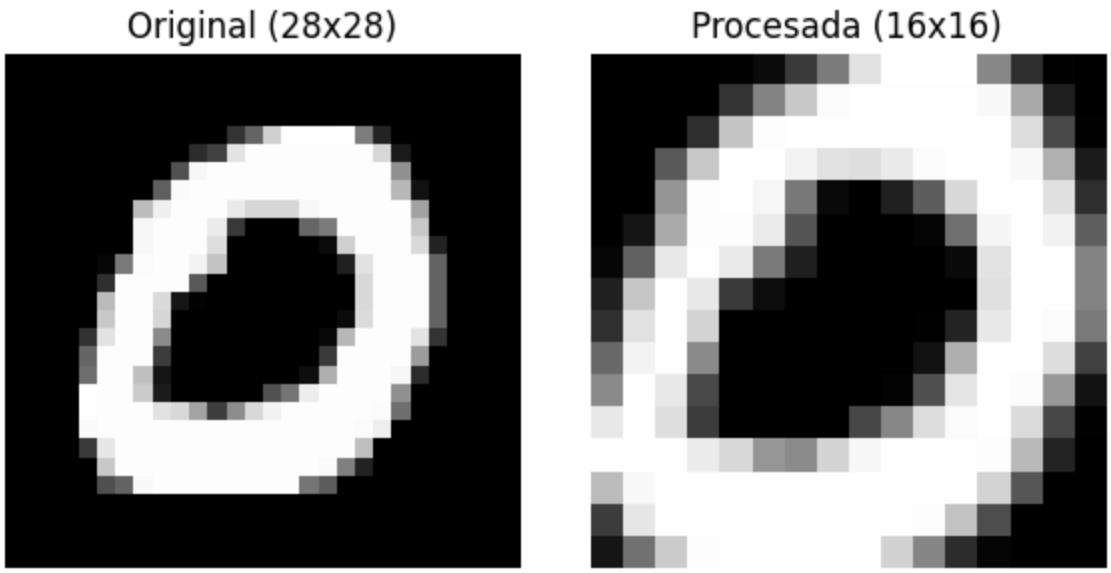
Porque usa promedio en lugar de eliminación: al reducir una imagen (downsampling), métodos básicos simplemente descartan píxeles, lo que podría "romper" o borrar las líneas delgadas de un dígito. INTER_AREA no elimina, sino que calcula un promedio ponderado de

los píxeles que caen dentro de la nueva cuadrícula de 16x16. Al promediar áreas, se mantiene intacta la "masa" geométrica y la continuidad del trazo. Algebraicamente, esto garantiza que no perdamos la información original (la varianza) del número, lo cual es vital para que la SVD encuentre las bases latentes correctas en los siguientes pasos.

In [4]:
```
# Diccionario para guardar las imágenes ya procesadas (recortadas y de 16x16)
imagenes_procesadas_k = {}

for d in range(10):
    lista_imagenes_procesadas = []

    for img in imagenes_crudas_k[d]:
        # 1. RECORTAR LA IMAGEN
        # Encontramos los índices de las filas y columnas que tienen al menos un pi
        filas_activas = np.where(img.max(axis=1) > 0)[0]
        cols_activas = np.where(img.max(axis=0) > 0)[0]

        # Validamos que la imagen no esté vacía (por seguridad)
        if len(filas_activas) > 0 and len(cols_activas) > 0:
            # Extraemos la sub-matriz que contiene solo el dígito
            recorte = img[filas_activas[0]:filas_activas[-1]+1,
                          cols_activas[0]:cols_activas[-1]+1]
        else:
            recorte = img

        # 2. REESCALAR A 16x16
        # cv2.INTER_AREA es el método matemático más recomendado al reducir dimensi
        reescalada = cv2.resize(recorte, (16, 16), interpolation=cv2.INTER_AREA)

        # normalizamos los valores entre 0 y 1 (paso ampliamente recomendado para t
        reescalada_norm = reescalada / 255.0

        lista_imagenes_procesadas.append(reescalada_norm)

    # Guardamos la lista de matrices 16x16 en el diccionario
    imagenes_procesadas_k[d] = lista_imagenes_procesadas
```

In [5]:
```
# --- Visualización del antes y después ---
# Tomemos el primer dígito (0) y su primera muestra [0] para comparar
img_original = imagenes_crudas_k[0][0]
img_procesada = imagenes_procesadas_k[0][0]

fig, axes = plt.subplots(1, 2, figsize=(6, 3))

axes[0].imshow(img_original, cmap='gray')
axes[0].set_title(f"Original (28x28)")
axes[0].axis('off')

axes[1].imshow(img_procesada, cmap='gray')
axes[1].set_title(f"Procesada (16x16)")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

```
# Verificación de dimensiones
print(f"Dimensión de la matriz original: {img_original.shape}")
print(f"Dimensión de la matriz procesada: {img_procesada.shape}")
```




```
Dimensión de la matriz original: (28, 28)
Dimensión de la matriz procesada: (16, 16)
```

> - **Finalmente, convierte cada imagen reescalada a un vector de longitud 256.**

¿Para qué aplanar cada matriz 16×16 a un vector de longitud 256?

Para finalizar la fase de preprocesamiento, debemos transformar nuestra representación espacial en una representación puramente algebraica. Tomamos cada matriz bidimensional de 16×16 y concatenamos sus filas una detrás de otra para formar un único vector unidimensional. Geométricamente, esto significa que dejamos de tratar la imagen como una cuadrícula de píxeles y pasamos a entender cada imagen como un único vector (o punto) que habita en el espacio Euclidiano de 256 dimensiones ($\mathbb{R}^{256}$). Con este cambio, podremos desarrollar las operaciones de álgebra lineal de los siguientes puntos (como las proyecciones ortogonales por ejemplo).

In [6]:
```
# Diccionario para guardar exclusivamente los vectores aplanados
vectores_k = {}

for d in range(10):
    lista_vectores = []

    # Iteramos sobre las imágenes ya procesadas (16x16)
    for img_16x16 in imagenes_procesadas_k[d]:
        # Aplanar la matriz a un vector 1D de longitud 256
        vector = img_16x16.flatten() # .flatten() toma la matriz de cada imagen y l
        lista_vectores.append(vector) #Este append nos servirá para que, al final d
```

```
    # Guardamos la lista de vectores para este dígito
    vectores_k[d] = lista_vectores

# --- Control de Calidad ---
# Tomamos el primer vector del dígito 0 para verificar la transformación
vector_prueba = vectores_k[0][0]

print(f"Total de clases (dígitos) procesadas: {len(vectores_k)}")
print(f"Cantidad de vectores para el dígito 0: {len(vectores_k[0])} (esto es 'k')")
print(f"Dimensiones de un solo vector: {vector_prueba.shape}")

# Verificación matemática (16 * 16 = 256)
assert vector_prueba.shape[0] == 256, "Error: El vector no tiene exactamente 256 di
```

```
Total de clases (dígitos) procesadas: 10
Cantidad de vectores para el dígito 0: 1226 (esto es 'k')
Dimensiones de un solo vector: (256,)
```

Análisis de la Estructura Resultante (vectores_k):

En este punto, nuestro conjunto de datos ha dejado de ser una colección visual de imágenes para convertirse formalmente en un conjunto de vectores algebraicos listos para el análisis de subespacios.

El objeto vectores_k almacena nuestro conjunto de entrenamiento estructurado de la siguiente manera:

- **Claves (Clases)**: Los números enteros del 0 al 9, representando cada uno de los dígitos a clasificar.
- **Valores (Muestras)**: Una lista que contiene exactamente k elementos para cada clase.
- **Elementos (Vectores)**: Cada elemento de la lista es un vector unidimensional (arreglo numérico) perteneciente al espacio euclidiano $\mathbb{R}^{256}$.

Esta estructura agrupa las muestras (vectores -> representan imagenes -> representan digitos) por clase (Numeros del 0 al 9) de tal manera que facilita el siguiente paso algebraico: tomar los k vectores individuales de cada dígito y concatenarlos horizontalmente para ensamblar las matrices de entrenamiento $M_d \in \mathbb{R}^{256\times k}$.

2.2. Construcción de Bases Latentes:

> - Para cada dígito d, organiza los k vectores resultantes en una matriz M_d (donde cada columna corresponde a una imagen).

En este paso, tomamos los vectores individuales (imágenes aplanadas) que preparamos en la fase anterior y los agrupamos por clase. Es decir, construimos 10 matrices en total (una para cada dígito del 0 al 9). Para cada clase d, ensamblamos una matriz $M_d \in \mathbb{R}^{256\times k}$ concatenando horizontalmente sus k vectores de entrenamiento.

¿Por qué concatenar horizontalmente los k vectores en M_d haciendo que cada imagen sea una columna?

Aunque en el análisis de datos tradicional (como en un DataFrame tabular) acostumbramos a que las filas sean las observaciones y las columnas las variables, en el álgebra lineal pura la convención y la operatividad cambian. Aquí representamos cada imagen entera como un vector columna para poder aprovechar las propiedades de los subespacios.

Geométricamente, esto equivale a tener k vectores (imagenes) flotando en un espacio de 256 dimensiones (donde cada eje representa un píxel). Como estos k vectores son representaciones del mismo dígito, sus coordenadas son muy similares (por ejemplo, todos los "ceros" tienen intensidades altas o bajas en las mismas zonas del lienzo). Por lo tanto, no apuntan de forma aleatoria, sino que forman un clúster que apunta hacia una región muy específica de ese espacio.

De esta manera, a partir de todas las combinaciones lineales posibles del conjunto de k vectores columna de un dígito, podremos generar luego lo que se conoce como el Espacio Columna de la matriz M_d. A esta región multidimensional la denominamos formalmente como el Subespacio U_d . Este subespacio representa entonces el "universo" geométrico donde habitan todas las variaciones posibles de los trazos de ese dígito en particular.

En conclusión: al ubicar nuestras imágenes como columnas, estamos declarando que el espacio columna de la matriz M_d define al subespacio vectorial U_d. Si hubiésemos acostado las imágenes como filas, las operaciones del álgebra lineal ya no mezclarían vectores que representan imágenes enteras, sino que intentarían mezclar vectores de posiciones aisladas (por ejemplo, combinar el "píxel número 1 de todas las fotos" con el "píxel número 2"). Eso crearía un espacio geométrico sin sentido, destruyendo el universo coherente de trazos que necesitamos para clasificar cada dígito.

In [7]:
```
# Diccionario para almacenar las matrices Md de cada clase
Md_matrices = {}

for d in range(10):
    # 1. Extraemos la lista de k vectores que le pertenece a este dígito
    lista_vectores_d = vectores_k[d]

    # 2. Apilamos los vectores como columnas para formar la matriz Md
    # np.column_stack toma la lista de (k) arreglos 1D y crea una matriz 2D de (256
    Md = np.column_stack(lista_vectores_d)

    # 3. Guardamos la matriz terminada en nuestro diccionario
    Md_matrices[d] = Md

# --- Control de Calidad ---
print(f"Total de matrices M_d generadas: {len(Md_matrices)}")
print(f"Dimensiones de la matriz M_0 (Dígito 0): {Md_matrices[0].shape}")
print(f"Dimensiones de la matriz M_8 (Dígito 8): {Md_matrices[8].shape}")
```

```
# Verificación de integridad
assert Md_matrices[0].shape == (256, len(vectores_k[0])), "Error en las dimensiones
```

```
Total de matrices M_d generadas: 10
Dimensiones de la matriz M_0 (Dígito 0): (256, 1226)
Dimensiones de la matriz M_8 (Dígito 8): (256, 1226)
```

> - Para cada dígito calcula una base orto-normal del espacio vectorial generado por las columnas de M_d, denotemos a esa base como B_d y llamemos a ese espacio generado U_d.

En el paso anterior construimos el universo posible para cada dígito agrupando k imágenes en la matriz M_d. Sin embargo, tener k columnas nos presenta un problema geométrico y computacional: la redundancia. Si seleccionáramos $k = 5000$, tendríamos 5000 imágenes del dígito "0", y es muy probable que muchas de ellas sean casi idénticas. En términos de álgebra lineal, esto significaría que tendríamos muchos vectores linealmente dependientes apuntando casi hacia la misma dirección, aportando información repetida. Para resolver esto y poder realizar proyecciones más limpias en etapas posteriores, no necesitamos los k vectores originales, sino una Base Ortonormal (B_d). Es decir, el conjunto mínimo indispensable de vectores que nos permita reconstruir cualquier forma del dígito "0" mediante combinaciones lineales.

En esta base orto-normal exigimos que:

- Ortogonal: Los vectores sean completamente independientes entre sí (formen ángulos exactos de 90 grados). Si un vector captura la "inclinación" del cero, otro capturará exclusivamente el "grosor", sin mezclar información.
- Normal: Todos los vectores tengan una longitud (norma) exactamente igual a 1. Esto estandariza la escala para que luego las proyecciones no se distorsionen.

Una vez halladas estas bases ortonormales para cada dígito (B_d), tendremos los ejes perfectos que definen el subespacio (U_d) de cada número.

¿Cómo transformar entonces nuestra matriz M_d en una base ortonormal B_d?

En lugar de usar el método de Gram-Schmidt (que puede ser numéricamente inestable con tantas dimensiones), usaremos la Descomposición en Valores Singulares (SVD). En esencia, la SVD toma nuestra matriz M_d y la factoriza en tres matrices matemáticas: $M_d = U\Sigma V^T$. Cada una cumple un rol específico en esta factorización:

- U (Los Trazos Maestros): Contiene los vectores singulares izquierdos. Sus columnas conforman una base ortonormal perfecta para el Espacio Columna de M_d. De aquí extraeremos directamente nuestra base B_d.
- Σ (La importancia): Es una matriz diagonal con los "valores singulares". Nos indica cuánta información útil o varianza aporta cada trazo maestro de U, ordenados de

mayor a menor importancia.

- V^T (Las Instrucciones de Mezcla): Contiene los vectores singulares derechos. Funciona como la "receta" que indica en qué proporciones exactas se deben mezclar los trazos de U para volver a reconstruir las k imágenes originales (con todo y su ruido).

Para nuestro objetivo de clasificación, nos quedaremos exclusivamente con U. Así, gracias a la SVD, habremos pasado de tener k imágenes ruidosas y redundantes (M_d) a tener una matriz impecable (B_d) con los vectores puros, ortogonales y de longitud 1 que definen el subespacio U_d (la esencia del dígito).

In [8]:
```
import numpy as np

# Diccionario para almacenar las bases ortonormales Bd de cada clase
Bd_bases = {}

for d in range(10):
    # 1. Tomamos la matriz Md del dígito d (nuestros datos crudos y ruidosos)
    Md = Md_matrices[d]

    # 2. Aplicamos la Descomposición en Valores Singulares (SVD)
    # full_matrices=False le dice a NumPy que nuestra matriz no es simetrica y que
    # Es decir, solo nos interesan las k columnas que definen nuestro subespacio,
    # ignorando las dimensiones vacías adicionales.
    U, S, Vt = np.linalg.svd(Md, full_matrices=False)

    # 3. La matriz U contiene nuestra base ortonormal pura. La bautizamos como Bd.
    Bd = U

    # 4. Guardamos la base terminada en nuestro diccionario
    Bd_bases[d] = Bd
```

In [9]:
```
# --- Control de Calidad y Pruebas Matemáticas ---
print("--- RESUMEN DE LA EXTRACCIÓN DE BASES ---")
print(f"Total de bases B_d generadas en el diccionario: {len(Bd_bases)}\n")

print("--- PRUEBA DE ORTONORMALIDAD POR DÍGITO ---")
# Variable para rastrear si todas pasaron la prueba
todas_pasaron = True

for d in range(10):
    # 1. Extraemos las matrices del dígito actual
    M_d = Md_matrices[d]
    B_d = Bd_bases[d]

    # 2. Prueba matemática: B^T * B = Matriz Identidad
    identidad_aprox = np.dot(B_d.T, B_d)
    # Comprobamos si es casi igual a la Identidad (tolerando mínimos errores de PC)
    es_ortonormal = np.allclose(identidad_aprox, np.eye(identidad_aprox.shape[0]),

    # 3. Formateamos el mensaje de éxito o error
    if es_ortonormal:
        estado = "✅ 100% Ortonormal"
    else:
```

```
        estado = "❌ ERROR"
        todas_pasaron = False

    # 4. Imprimimos el reporte para este dígito
    print(f"Dígito {d} | Matriz original: {M_d.shape} ➡️ Base ortonormal generada:

# Conclusión general
if todas_pasaron:
    print("\n CONCLUSIÓN: Las 10 bases superaron la prueba matemática.")
else:
    print("\n⚠️ ADVERTENCIA: Alguna de las bases falló la prueba de ortonormalidad
```

```
--- RESUMEN DE LA EXTRACCIÓN DE BASES ---
Total de bases B_d generadas en el diccionario: 10

--- PRUEBA DE ORTONORMALIDAD POR DÍGITO ---
Dígito 0 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 1 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 2 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 3 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 4 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 5 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 6 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 7 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 8 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal
Dígito 9 | Matriz original: (256, 1226) ➡️ Base ortonormal generada: (256, 256) | P
rueba: ✅ 100% Ortonormal

 CONCLUSIÓN: Las 10 bases superaron la prueba matemática.
```

Nota sobre el Límite Dimensional del Subespacio

Al observar las dimensiones resultantes de nuestra SVD, notamos que le entregamos al algoritmo una matriz M_0 con **1226 columnas** (imágenes), pero nos devolvió una base B_0 con exactamente **256 columnas** (vectores).

¿Por qué la base ortonormal no supera las 256 dimensiones si le dimos 1226 muestras?

La respuesta reside en la **Regla de la Dimensionalidad** del álgebra lineal. Nuestras imágenes originales tienen un tamaño de 16×16 píxeles, lo que significa que al aplanarlas, habitan en un espacio vectorial de exactamente 256 dimensiones ($\mathbb{R}^{256}$).

Geométricamente, es imposible encontrar más de N vectores mutuamente ortogonales dentro de un espacio de N dimensiones . Así como en una habitación 3D solo caben 3 ejes

perpendiculares (X, Y, Z), en nuestro "universo de imágenes" solo caben 256 ejes perpendiculares.

Cuando la SVD procesó nuestras 1226 imágenes, encontró los 256 "ejes maestros" absolutos que sostienen todo el subespacio. En ese punto, el espacio se "llenó". El algoritmo detectó correctamente que las 970 imágenes restantes no aportaban ninguna dirección geométrica nueva; eran linealmente dependientes y podían construirse perfectamente como combinaciones de los primeros 256 vectores. Por lo tanto, extrajo la esencia pura y descartó la redundancia extrema.

Optimización Adicional: SVD Truncada y Varianza Explicada

Aunque nuestra base B_d de 256 dimensiones captura matemáticamente todo el subespacio, sabemos que no todos esos 256 vectores son igualmente importantes. Los primeros vectores capturan los "trazos maestros" del dígito, mientras que los últimos probablemente solo modelen el ruido de los datos (píxeles erróneos, malos pulsos al escribir).

Para hacer nuestro modelo más eficiente y evitar el sobreajuste (*overfitting* al ruido), podemos truncar nuestra base, quedándonos únicamente con un número menor de vectores ($r < 256$) que logren retener la mayor parte de la información útil.

Para decidir este "punto de corte", analizaremos la matriz Σ (valores singulares). La fracción de **Varianza Explicada Acumulada** se calcula usando los cuadrados de los valores singulares:

$$\text{Varianza Acumulada} = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_{i=1}^{256} \sigma_i^2}$$

Análisis Geométrico: Identificación del "Codo" Real

En lugar de imponer un umbral arbitrario de varianza (como el 90%), realizaremos un análisis visual de la curvatura en cada

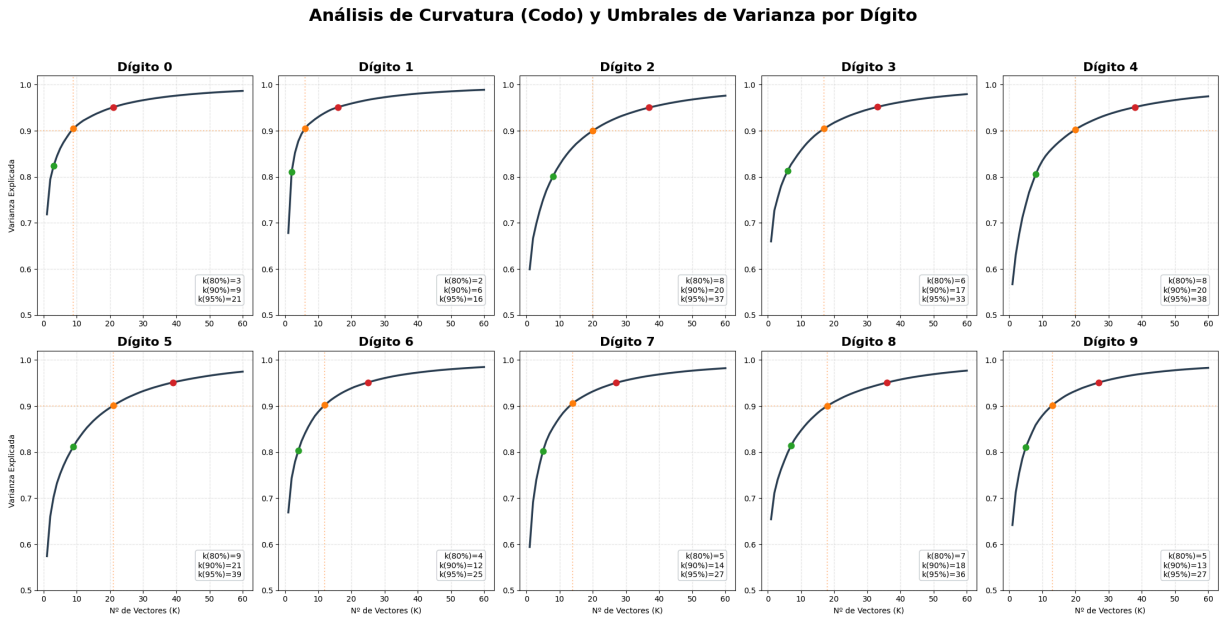
Siguiendo la Clase 4, el **codo** es el punto donde la magnitud de los valores singulares comienza a aplanarse significativamente. Este punto es crítico porque marca la frontera entre:

- **La Señal:** Los primeros componentes que capturan la estructura geométrica esencial y común del dígito.
- **El Ruido:** Los componentes posteriores que solo capturan variaciones aleatorias o imperfecciones de los datos originales.

Al observar los 10 gráficos simultáneamente, buscaremos el punto de máxima curvatura. Esto nos permitirá decidir un k que respete la geometría intrínseca de cada clase, en lugar de forzar un porcentaje estadístico:

In [10]:
```
import matplotlib.pyplot as plt
import numpy as np

# Configuración de la figura para que sea legible y estética
fig, axes = plt.subplots(2, 5, figsize=(22, 12))
fig.suptitle('Análisis de Curvatura (Codo) y Umbrales de Varianza por Dígito', font

# Diccionario para guardar los hallazgos y comparar al final
analisis_k = {}

for d in range(10):
    M_d = Md_matrices[d]
    _, S_d, _ = np.linalg.svd(M_d, full_matrices=False)

    # Varianza acumulada
    varianza_acum = np.cumsum(S_d**2) / np.sum(S_d**2)

    # Encontramos los hitos matemáticos
    k80 = np.argmax(varianza_acum >= 0.80) + 1
    k90 = np.argmax(varianza_acum >= 0.90) + 1
    k95 = np.argmax(varianza_acum >= 0.95) + 1
    analisis_k[d] = [k80, k90, k95]

    row, col = d // 5, d % 5
    ax = axes[row, col]

    # 1. Graficamos la curva principal (los primeros 60 componentes para ver bien e
    rango_visual = range(1, 61)
    ax.plot(rango_visual, varianza_acum[:60], color='#2c3e50', linewidth=2.5, label

    # 2. Marcamos los puntos de control con colores distintos
    ax.scatter(k80, varianza_acum[k80-1], color='tab:green', s=60, zorder=5, label=
    ax.scatter(k90, varianza_acum[k90-1], color='tab:orange', s=60, zorder=5, label
    ax.scatter(k95, varianza_acum[k95-1], color='tab:red', s=60, zorder=5, label='9

    # 3. Añadimos líneas guía muy sutiles para ver la proyección hacia los ejes
    ax.axvline(x=k90, color='tab:orange', linestyle=':', alpha=0.4)
    ax.axhline(y=0.90, color='tab:orange', linestyle=':', alpha=0.4)

    # Estética del subgráfico
    ax.set_title(f'Dígito {d}', fontsize=16, fontweight='bold')
    ax.set_ylim(0.5, 1.02)
    ax.grid(True, linestyle='--', alpha=0.3)

    # Leyenda interna compacta con los valores de K
    texto_k = f"k(80%)={k80}\nk(90%)={k90}\nk(95%)={k95}"
    ax.text(0.95, 0.05, texto_k, transform=ax.transAxes, fontsize=10,
            verticalalignment='bottom', horizontalalignment='right',
            bbox=dict(boxstyle='round', facecolor='white', alpha=0.8, edgecolor='#b

    if row == 1: ax.set_xlabel('Nº de Vectores (K)')
    if col == 0: ax.set_ylabel('Varianza Explicada')

plt.tight_layout(rect=[0, 0.03, 1, 0.92])
plt.show()
```

```
# --- TABLA RESUMEN PARA COMPARAR COMPLEJIDAD ---
print(f"{'Dígito':<8} | {'k(80%)':<8} | {'k(90%)':<8} | {'k(95%)':<8} | {'Complejid
print("-" * 60)
for d, ks in analisis_k.items():
    # Una métrica simple de complejidad: a mayor k para llegar al 90%, más complejo
    comp = "Alta" if ks[1] > 12 else "Baja" if ks[1] < 7 else "Media"
    print(f"{d:<8} | {ks[0]:<8} | {ks[1]:<8} | {ks[2]:<8} | {comp}")
```




```
Dígito   | k(80%)   | k(90%)   | k(95%)   | Complejidad
------------------------------------------------------------
0        | 3        | 9        | 21       | Media
1        | 2        | 6        | 16       | Baja
2        | 8        | 20       | 37       | Alta
3        | 6        | 17       | 33       | Alta
4        | 8        | 20       | 38       | Alta
5        | 9        | 21       | 39       | Alta
6        | 4        | 12       | 25       | Media
7        | 5        | 14       | 27       | Alta
8        | 7        | 18       | 36       | Alta
9        | 5        | 13       | 27       | Alta
```

Interpretación del Análisis de Varianza y Scree Plot:

Tras realizar un análisis comparativo de la varianza explicada acumulada para los diez dígitos, observamos que la complejidad geométrica no es uniforme. Mientras que el dígito "1" es altamente eficiente (alcanza el 90% de varianza con solo 6 vectores), dígitos más complejos como el "5" requieren hasta 21 vectores para capturar el mismo nivel de información.

Este análisis nos permite identificar el "codo" de las curvas: el punto de máxima curvatura donde la ganancia de información deja de ser estructural y comienza a ser marginal. Observamos que:

- La Señal: Se concentra en los primeros componentes (donde la curva asciende verticalmente). Aquí reside la forma arquetípica del número.

- El Ruido: Aparece tras el codo, donde la curva se aplana. Estos vectores adicionales capturan variaciones aleatorias, grosores de trazo específicos o ruido en los píxeles que no ayudan a la generalización del modelo.

Decisión Estratégica:

Aunque disponemos de bases ortonormales de 256 dimensiones, utilizarlas completas implicaría proyectar nuestras imágenes sobre ejes que son mayoritariamente ruido. Para evitar el sobreajuste y optimizar nuestro algoritmo, adoptaremos un enfoque de SVD Truncada con un valor global de $k_{trunc} = 21$. Esta decisión se basa en el "peor caso" observado (el dígito 5), garantizando que todos los subespacios retengan al menos el 90% de su varianza explicada, asegurando que no perdamos la esencia de los números más complejos.

Nota sobre la notación: Para evitar ambigüedades en el código y en el análisis, a partir de este punto distinguiremos explícitamente entre k_{trunc} (el número de vectores ortonormales retenidos en la SVD, que fijamos en 21) y k_{train} (el tamaño de la muestra de imágenes de entrenamiento por dígito).

> - ¿Cómo se puede interpretar el espacio generado por B_d en palabras?

En adición a las interpretaciones previas sobre el espacio generado por B_d, podríamos interpretarlo como la representación minimalista del dígito d. Es decir, como el subespacio donde habitan todas las formas "posibles" o "normalmente utilizadas" para escribir ese número en particular. B_d nos permite ignorar el ruido o los detalles que no aportan mucha información de tal manera que nos permite centrar la atención en aquellas características del dígito que mejor lo definen matemáticamente.

Al ser una base ortonormal, define un universo donde cada eje controla de forma independiente una característica clave del trazo (como la inclinación o el grosor), eliminando cualquier redundancia. De cara al ejercicio de clasificación, si una imagen nueva "encaja" bien al proyectarse sobre este espacio, significa que comparte la misma lógica geométrica que destilamos de los datos de entrenamiento; de lo contrario, el residuo será alto y sabremos que no pertenece a esa categoría.

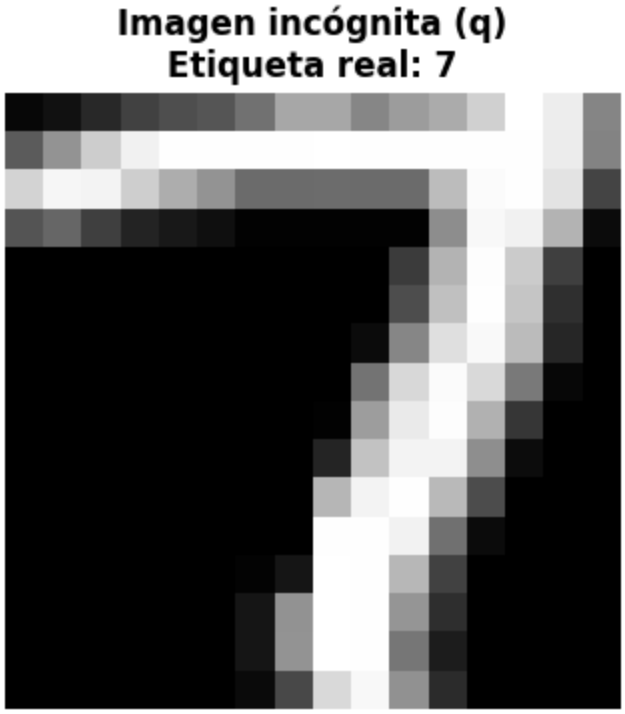
2.3. Clasificación de una Imagen Nueva:

> - Toma una imagen de prueba (que no esté en el conjunto de entrenamiento) y apliquele el mismo proceso: recortar, reescalar a 16×16 y convertir a un vector q de longitud 256.

In [11]:
```
import random
import cv2
```

```python
import numpy as np
import matplotlib.pyplot as plt


# 1. Seleccionamos un índice aleatorio de todo el conjunto de PRUEBA
np.random.seed(16)
indice_aleatorio = np.random.randint(0, len(X_test_full))

# 2. Extraemos la imagen cruda y su etiqueta real
img_prueba = X_test_full[indice_aleatorio]
etiqueta_real = y_test_full[indice_aleatorio]

# --- 3. REPLICAMOS EL PROCESAMIENTO EXACTO DEL PUNTO 2.1 ---

# A. RECORTAR LA IMAGEN
filas_activas = np.where(img_prueba.max(axis=1) > 0)[0]
cols_activas = np.where(img_prueba.max(axis=0) > 0)[0]

if len(filas_activas) > 0 and len(cols_activas) > 0:
    recorte = img_prueba[filas_activas[0]:filas_activas[-1]+1,
                         cols_activas[0]:cols_activas[-1]+1]
else:
    recorte = img_prueba

# B. REESCALAR A 16x16
reescalada = cv2.resize(recorte, (16, 16), interpolation=cv2.INTER_AREA)

# C. NORMALIZAR (0 a 1)
reescalada_norm = reescalada / 255.0

# D. VECTORIZAR (Aplanar a vector de 256 posiciones)
q = reescalada_norm.flatten()

# --- Control de Calidad y Visualización ---
print(f"--- PREPARACIÓN DEL VECTOR DE PRUEBA ---")
print(f"Índice seleccionado en X_test_full: {indice_aleatorio}")
print(f"Etiqueta real de la imagen oculta: {etiqueta_real}")
print(f"Dimensiones del vector q: {q.shape}")

# Graficamos el vector q (volviendo a darle forma 16x16 solo para la visualización)
plt.figure(figsize=(4, 4))
plt.imshow(q.reshape(16, 16), cmap='gray')
plt.title(f"Imagen incógnita (q)\nEtiqueta real: {etiqueta_real}", fontweight='bold
plt.axis('off')
plt.show()
```

```
--- PREPARACIÓN DEL VECTOR DE PRUEBA ---
Índice seleccionado en X_test_full: 6825
Etiqueta real de la imagen oculta: 7
Dimensiones del vector q: (256,)
```



> - Para cada dígito d, calcula la proyección ortogonal de q sobre el subespacio generado por B_d, obteniendo el vector proyectado p_d.

Para averiguar a qué dígito se parece más nuestra imagen incógnita q, debemos proyectar este vector sobre los 10 subespacios generados por nuestras bases B_d.

Siguiendo nuestro análisis de varianza previo, aplicaremos una Aproximación de Bajo Rango (SVD Truncada) con $k_{trunc} = 21$. Por lo tanto, no utilizaremos los 256 vectores de la base completa, sino únicamente una submatriz U_d compuesta por sus primeras 21 columnas. Esto garantiza que proyectemos q sobre la "señal pura" estructural y evitemos el sobreajuste al ruido.

Como vimos en clase, la matriz de proyección general es $P = A(A^T A)^{-1} A^T$. Sin embargo, dado que las columnas de nuestra matriz U_d son ortonormales por construcción, se cumple que $U_d^T U_d = I$, lo que simplifica la fórmula a:

$$p_d = U_d(U_d^T q)$$

A nivel matricial, esta operación realiza dos procesos de transformación de datos:

1. Compresión y Extracción de Características $(U_d^T \times q)$**:**
Nuestra matriz U_d (el molde truncado) tiene dimensiones `(256, 21)`. Al transponerla a U_d^T, la "acostamos" a `(21, 256)`. Al multiplicar esta matriz por nuestra imagen incógnita q, que es un vector columna `(256, 1)`, las dimensiones internas coinciden perfectamente. El resultado es un vector hiper-comprimido de tamaño `(21, 1)`. **Significado Analítico:** En este paso abandonamos el espacio de los píxeles. Pasamos de tener 256 puntos "crudos" y

sin contexto, a extraer un resumen preciso de 21 "coordenadas". Una forma de interpretar este nuevo vector es verlo como un escaneo de compatibilidad: cada uno de sus 21 números nos indica el "peso" exacto o el nivel de activación de los patrones macro-geométricos (vectores ortogonales) que componen al dígito contra el cual nos estamos comparando.

2. Reconstrucción Forzada $(U_d \times \text{coordenadas})$**:**
Ahora tomamos nuestra matriz original U_d (el molde truncado), que mantiene sus dimensiones de `(256, 21)`, y la multiplicamos por el vector hiper-comprimido de coordenadas que acabamos de obtener, cuyo tamaño es `(21, 1)`. Nuevamente, las dimensiones internas encajan a la perfección. El resultado matemático de esta operación nos devuelve al espacio original: obtenemos un nuevo vector columna de tamaño `(256, 1)`. Este es nuestro vector proyectado p_d. **Significado Analítico**: En este paso fabricamos una imagen artificial desde cero. Le entregamos al modelo los 21 "pesos" calculados en la etapa anterior para que actúen como una receta de ensamblaje. Le estamos ordenando: "Ve al molde U_d, toma sus 21 patrones macro-geométricos y mézclalos en las proporciones exactas que dicta esta receta para redibujar la imagen en el lienzo completo de 256 píxeles".

El poder predictivo del clasificador reside aquí: como el molde U_d solo conoce las reglas para dibujar un dígito específico, forzará la reconstrucción para que siga exclusivamente esas reglas. Si la imagen original q pertenecía a ese dígito, el molde la entenderá y la reconstrucción (p_d) será muy fiel a la original. Si era otro número, el molde fracasará en su intento de dibujarla con sus herramientas limitadas, generando una reconstrucción defectuosa

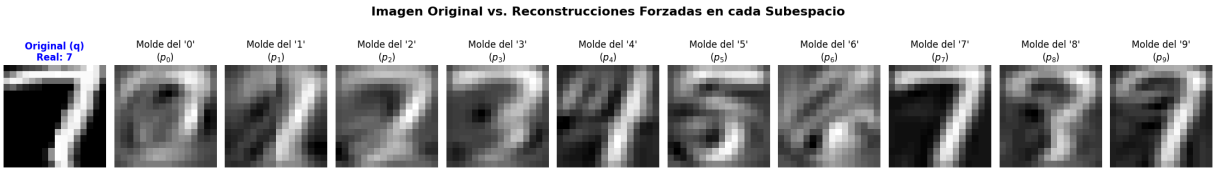
In [12]:
```
# Definimos nuestro punto de corte estratégico basado en el análisis de varianza
k_trunc = 21

# Diccionario para guardar el vector proyectado p_d para cada dígito
proyecciones_pd = {}

for d in range(10):
    # 1. Tomamos la base ortonormal completa de este dígito
    B_d = Bd_bases[d]

    # 2. TRUNCAMIENTO: Nos quedamos solo con las primeras 21 columnas (los trazos m
    U_d = B_d[:, :k_trunc]

    # 3. PROYECCIÓN ORTOGONAL (Aplicando la fórmula simplificada de la Clase 4)
    # Primero encontramos las coordenadas de q en el subespacio
    coordenadas_subespacio = np.dot(U_d.T, q)

    # Luego reconstruimos el vector proyectado p_d en el espacio 256D
    p_d = np.dot(U_d, coordenadas_subespacio)

    # Guardamos el resultado
    proyecciones_pd[d] = p_d

# --- Control de Calidad ---
print(f"--- PROYECCIONES ORTOGONALES CALCULADAS ---")
```

```
print(f"Se generaron las proyecciones p_d para los 10 dígitos.")
print(f"Bases truncadas utilizadas (U_d): {U_d.shape}")
print(f"Dimensiones de cada vector proyectado p_d: {proyecciones_pd[0].shape}")
```

```
--- PROYECCIONES ORTOGONALES CALCULADAS ---
Se generaron las proyecciones p_d para los 10 dígitos.
Bases truncadas utilizadas (U_d): (256, 21)
Dimensiones de cada vector proyectado p_d: (256,)
```

Veamos graficamente como se reconstruyeron los vectores proyectados p_d en cada subespacio:

In [13]:
```
import matplotlib.pyplot as plt

# Creamos una figura ancha para poner la imagen original y sus 10 proyecciones lado
fig, axes = plt.subplots(1, 11, figsize=(22, 3))
fig.suptitle('Imagen Original vs. Reconstrucciones Forzadas en cada Subespacio', fo

# 1. Graficamos la imagen incógnita original (q)
axes[0].imshow(q.reshape(16, 16), cmap='gray')
axes[0].set_title(f"Original (q)\nReal: {etiqueta_real}", color='blue', fontweight=
axes[0].axis('off')

# 2. Graficamos cómo quedó q al ser proyectada en cada uno de los 10 moldes
for d in range(10):
    p_d = proyecciones_pd[d]

    # Volvemos a darle forma de matriz 16x16 para poder verla como imagen
    imagen_reconstruida = p_d.reshape(16, 16)

    axes[d+1].imshow(imagen_reconstruida, cmap='gray')
    axes[d+1].set_title(f"Molde del '{d}'\n($p_{d}$)")
    axes[d+1].axis('off')

plt.tight_layout()
plt.show()
```



La visualización anterior indica que la reconstrucción p_7 es la más nítida, sólida y prácticamente idéntica a la original q. Esto significa que la "receta" de 21 coordenadas encontró en el subespacio del '7' los trazos rectos y diagonales necesarios para dibujarlo a la perfección.

Por otro lado, el subespacio del '0' que solo sabe dibujar curvas cerradas, intentó forzar el '7' a ser un óvalo, resultando en una mancha grisácea y borrosa. El Molde del '1' intentó capturar la diagonal, pero ignoró por completo la línea horizontal superior porque ese trazo "no existe" en su universo. el Molde del '9' y el '2', no se ven tan mal. Esto tiene sentido porque un '9' y un '7', e incluso un '2', comparten mucha estructura (una línea superior y una

diagonal descendente). El modelo intentó cerrarle un pequeño bucle arriba, pero sigue siendo una reconstrucción decente aunque menos nitida y fina en los detalles.

No obstante, como ya tenemos nuestra imagen original q y sus 10 reconstrucciones forzadas p_d. Geométricamente, sabemos que la reconstrucción más fiel será la que deje el menor residuo.

Para cuantificar esto, calcularemos la **distancia euclidiana** (la norma L_2) entre el vector original y cada vector proyectado, utilizando la fórmula vista en clase:

$$dist(q, d) = ||q - p_d||$$

Analíticamente, esta distancia representa la "cantidad de píxeles" (o la magnitud del error) que el molde U_d no pudo explicar ni reconstruir.

- Una distancia pequeña significa que q "encaja" muy bien en el subespacio (es decir, el molde tenía todas las reglas para dibujarlo).
- Una distancia grande significa que el subespacio carece de la información estructural para representar a q.

Finalmente, nuestro clasificador funcionará bajo un principio de **minimización del error**: le asignaremos a la imagen q la etiqueta del dígito d que arroje la menor distancia.

> - Define la distancia $dist(q, d) = ||q - p_d||$. Clasifica la imagen q asignándole la etiqueta del dígito d que minimice esta distancia.

In [14]:
```
import numpy as np

# Diccionario para guardar las distancias a cada subespacio
distancias = {}

print("--- CÁLCULO DE DISTANCIAS ||q - p_d|| ---")

# Calculamos la distancia euclidiana entre q y cada proyección p_d
for d in range(10):
    p_d = proyecciones_pd[d]

    # La norma L2 del vector residual (q - p_d)
    distancia_d = np.linalg.norm(q - p_d)
    distancias[d] = distancia_d

    print(f"Distancia al subespacio del '{d}': {distancia_d:.4f}")

# Encontramos la etiqueta con la menor distancia
etiqueta_predicha = min(distancias, key=distancias.get)
distancia_minima = distancias[etiqueta_predicha]

print("\n" + "="*40)
print(f"RESULTADO DE LA CLASIFICACIÓN")
print("="*40)
```

```
print(f"Etiqueta Real de la imagen : {etiqueta_real}")
print(f"Etiqueta Predicha (Min Dist): {etiqueta_predicha} (con distancia {distancia

if etiqueta_real == etiqueta_predicha:
    print("✅ ¡Hemos clasificado el dígito correctamente!")
else:
    print("❌ La clasificación falló.")
print("="*40)
```

```
--- CÁLCULO DE DISTANCIAS ||q - p_d|| ---
Distancia al subespacio del '0': 4.0971
Distancia al subespacio del '1': 3.5111
Distancia al subespacio del '2': 3.6624
Distancia al subespacio del '3': 3.2905
Distancia al subespacio del '4': 4.1658
Distancia al subespacio del '5': 4.6490
Distancia al subespacio del '6': 5.2028
Distancia al subespacio del '7': 1.3241
Distancia al subespacio del '8': 3.4954
Distancia al subespacio del '9': 2.8996

========================================
RESULTADO DE LA CLASIFICACIÓN
========================================
Etiqueta Real de la imagen : 7
Etiqueta Predicha (Min Dist): 7 (con distancia 1.3241)
✅ ¡Hemos clasificado el dígito correctamente!
========================================
```

El cálculo de las distancias $||q - p_d||$ nos ofrece una radiografía perfecta de cómo el modelo "razona" geométricamente. Al analizar el vector de resultados para nuestra imagen de prueba (un '7'), observamos patrones fascinantes que confirman la robustez del método de proyecciones ortogonales:

El modelo asignó la etiqueta **'7'** al obtener la distancia mínima absoluta de **1.3241**. Geométricamente, esto significa que el subespacio U_7 poseía casi todas las instrucciones (trazos horizontales superiores y diagonales descendentes) necesarias para reconstruir la imagen original q, dejando un residuo de píxeles (error) muy bajo.

Es sumamente revelador observar cuáles fueron los dígitos que quedaron en segundo y tercer lugar. Las distancias más cercanas al '7' fueron:

- **'9'** (Distancia: 2.8996)
- **'3'** (Distancia: 3.2905)
- **'8'** (Distancia: 3.4954)

Esto demuestra que el modelo no se equivoca de forma aleatoria. Un '7' manuscrito comparte muchas características con el '1' (la línea vertical/diagonal), y frecuentemente comparte la barra horizontal superior con el '9' y el '3' (e incluso el '8'). Los moldes de estos números lograron reconstruir *partes* del '7', pero fallaron en los detalles finos, arrojando distancias moderadas.

En el extremo opuesto, vemos distancias inmensas para los dígitos **'6'** (5.2028), **'5'** (4.6490) y **'4'** (4.1658). El molde del '6', por ejemplo, está dominado por vectores que dibujan bucles y círculos inferiores. Al intentar forzar la geometría recta y angular del '7' usando reglas circulares, el modelo dejó una enorme cantidad de píxeles sin explicar, lo que disparó matemáticamente la métrica de error.

2.4. Evaluación y Análisis:

> - Utiliza un conjunto de validación (imágenes de prueba distintas a las utilizadas para formar M_d) y evalúa el porcentaje de aciertos de tu clasificador.

Paso 1: Encapsulación del Algoritmo de Predicción

Para poder iterar masivamente sobre el conjunto de validación, primero consolidaremos todo el *pipeline* desarrollado en el Punto 2.3 en una única función de inferencia. Esta función tomará una imagen cruda (28x28) y devolverá la etiqueta predicha, ejecutando internamente:

1. El preprocesamiento (recorte, reescalado a 16x16 y vectorización a q).
2. La proyección ortogonal sobre los 10 subespacios truncados (U_d con $k_{trunc} = 21$).
3. El cálculo de la norma L_2 ($||q - p_d||$) para determinar el residuo mínimo.

In [15]:
```
import cv2
import numpy as np

def predecir_digito(imagen_cruda, diccionario_bases, k_trunc=21):
    """
    Toma una imagen cruda de MNIST, la proyecta sobre los subespacios
    y retorna el dígito predicho basado en la mínima distancia ortogonal.
    """
    # --- 1. PREPROCESAMIENTO ---
    filas_activas = np.where(imagen_cruda.max(axis=1) > 0)[0]
    cols_activas = np.where(imagen_cruda.max(axis=0) > 0)[0]

    if len(filas_activas) > 0 and len(cols_activas) > 0:
        recorte = imagen_cruda[filas_activas[0]:filas_activas[-1]+1,
                               cols_activas[0]:cols_activas[-1]+1]
    else:
        recorte = imagen_cruda

    reescalada = cv2.resize(recorte, (16, 16), interpolation=cv2.INTER_AREA)
    q = (reescalada / 255.0).flatten()

    # --- 2. PROYECCIÓN Y DISTANCIAS ---
    mejor_digito = -1
    distancia_minima = float('inf') # Inicializamos con infinito

    for d in range(10):
```

```
        # Extraemos el molde truncado
        U_d = diccionario_bases[d][:, :k_trunc]

        # Calculamos proyeccion y distancia
        coordenadas = np.dot(U_d.T, q)
        p_d = np.dot(U_d, coordenadas)
        distancia_actual = np.linalg.norm(q - p_d)

        # Actualizamos si encontramos un mejor ajuste
        if distancia_actual < distancia_minima:
            distancia_minima = distancia_actual
            mejor_digito = d

    return mejor_digito

# Prueba rápida de cordura con la imagen que usamos en el paso 2.3
# (Debería imprimir el mismo resultado: 7)
prediccion_prueba = predecir_digito(X_test_full[indice_aleatorio], Bd_bases, k_trun
print(f"Prueba de la función: Etiqueta real {y_test_full[indice_aleatorio]}, Predic
```

```
Prueba de la función: Etiqueta real 7, Predicción 7
```

Paso 2: Evaluemos sobre el Conjunto de Prueba Completo

Para medir el porcentaje de aciertos real de nuestro modelo geométrico, iteraremos sobre las 10,000 imágenes que componen el conjunto `X_test_full` de MNIST. Ninguna de estas imágenes fue utilizada durante la construcción de los subespacios (matrices M_d), por lo que representan datos completamente nuevos para el algoritmo.

Llevaremos un conteo de las predicciones correctas para calcular el *Accuracy* global. Adicionalmente, almacenaremos las predicciones para poder analizar más adelante en qué dígitos específicos se está confundiendo nuestro modelo.

In [16]:
```
import time

# Variables para el seguimiento del rendimiento
total_imagenes = len(X_test_full)
aciertos = 0
predicciones_totales = []

print(f"--- INICIANDO EVALUACIÓN MASIVA ({total_imagenes} imágenes) ---")
tiempo_inicio = time.time()

# Iteramos sobre todo el conjunto de prueba
for i in range(total_imagenes):
    imagen_cruda = X_test_full[i]
    etiqueta_real = y_test_full[i]

    # Usamos nuestra función encapsulada (truncando los vectores ortonormales a k=2
    prediccion = predecir_digito(imagen_cruda, Bd_bases, k_trunc=21)

    # Guardamos la predicción para análisis futuros
    predicciones_totales.append(prediccion)
```

```
    # Verificamos si acertó
    if prediccion == etiqueta_real:
        aciertos += 1

    # Barra de progreso simple cada 1000 imágenes
    if (i + 1) % 1000 == 0:
        print(f"Procesadas {i + 1}/{total_imagenes} imágenes... Aciertos parciales:

# --- RESULTADOS FINALES ---
tiempo_fin = time.time()
tiempo_total = tiempo_fin - tiempo_inicio
accuracy = (aciertos / total_imagenes) * 100

print("\n" + "="*50)
print(f"RESULTADOS DE LA EVALUACIÓN (SVD Truncada k_trunc=21)")
print("="*50)
print(f"Total de imágenes evaluadas: {total_imagenes}")
print(f"Predicciones correctas:      {aciertos}")
print(f"Predicciones incorrectas:    {total_imagenes - aciertos}")
print(f"Porcentaje de Aciertos (Accuracy): {accuracy:.2f}%")
print(f"Tiempo total de procesamiento: {tiempo_total:.2f} segundos")
print("="*50)
```

```
--- INICIANDO EVALUACIÓN MASIVA (10000 imágenes) ---
Procesadas 1000/10000 imágenes... Aciertos parciales: 939
Procesadas 2000/10000 imágenes... Aciertos parciales: 1883
Procesadas 3000/10000 imágenes... Aciertos parciales: 2828
Procesadas 4000/10000 imágenes... Aciertos parciales: 3771
Procesadas 5000/10000 imágenes... Aciertos parciales: 4704
Procesadas 6000/10000 imágenes... Aciertos parciales: 5671
Procesadas 7000/10000 imágenes... Aciertos parciales: 6640
Procesadas 8000/10000 imágenes... Aciertos parciales: 7620
Procesadas 9000/10000 imágenes... Aciertos parciales: 8599
Procesadas 10000/10000 imágenes... Aciertos parciales: 9567

==================================================
RESULTADOS DE LA EVALUACIÓN (SVD Truncada k_trunc=21)
==================================================
Total de imágenes evaluadas: 10000
Predicciones correctas:      9567
Predicciones incorrectas:    433
Porcentaje de Aciertos (Accuracy): 95.67%
Tiempo total de procesamiento: 2.47 segundos
==================================================
```

Análisis del Rendimiento del Modelo

La evaluación masiva sobre las 10,000 imágenes del conjunto de validación arrojó un porcentaje de aciertos del **95.67%**, ejecutado en apenas un par de segundos.

Este resultado es altamente significativo por dos razones:

1. **Poder de Generalización:** Al no haber usado estas imágenes para construir las matrices M_d, confirmamos que los subespacios realmente capturaron el "ADN geométrico" de los números y no se limitaron a memorizar los datos de entrenamiento (*overfitting*).

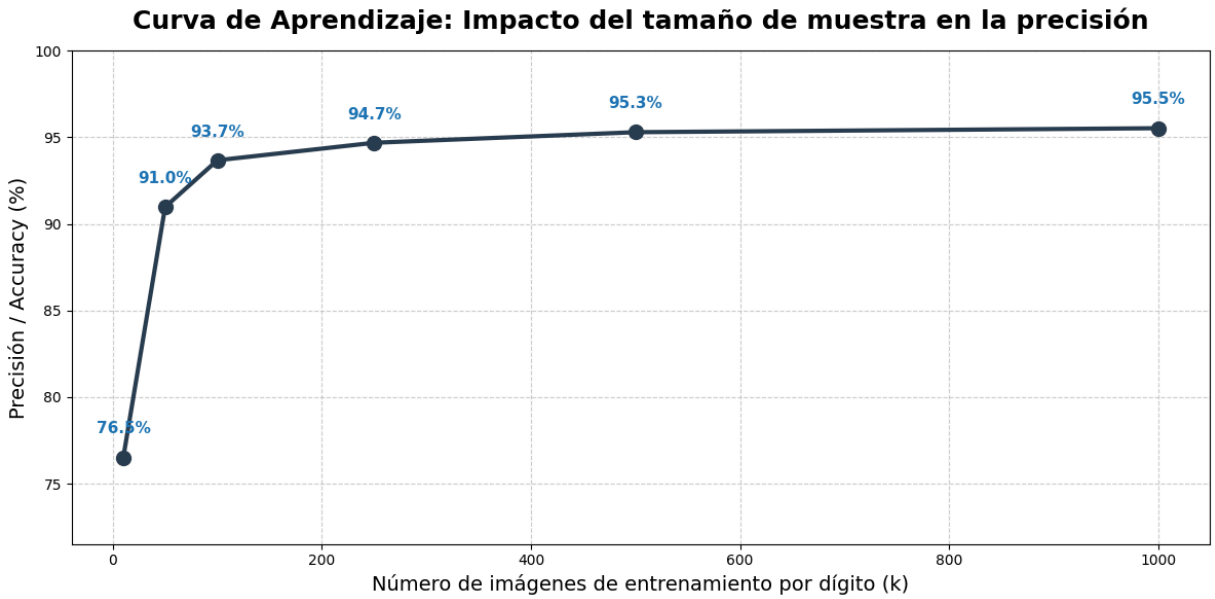
2. **Eficiencia Computacional:** El tiempo de procesamiento (aprox. 2 segundos para 10,000 imágenes) valida la decisión de usar la matriz identidad en la proyección ortogonal, transformando un problema costoso de inversión de matrices en un simple doble producto punto matricial rápido y altamente optimizado.

> - Repite la evaluación variando k (el número de imágenes de entrenamiento por dígito) y grafica la precisión del clasificador en función de k.

In [17]:
```
import time
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Definimos los diferentes tamaños de muestra (k) que queremos probar
valores_k_train = [10, 50, 100, 250, 500, 1000]
precisiones = []

print("--- INICIANDO EXPERIMENTO DE CURVA DE APRENDIZAJE ---")

for k_train in valores_k_train:
    print(f"Construyendo y evaluando modelo con k = {k_train} imágenes por dígito..
    tiempo_inicio_k = time.time()

    # 1. Construir las bases ortonormales para este tamaño de muestra
    bases_temporales = {}
    for d in range(10):
        # Filtramos las imágenes de la clase 'd'
        imagenes_clase_d = X_train_full[y_train_full == d]

        # Tomamos solo las primeras 'k_train' imágenes
        muestra_entrenamiento = imagenes_clase_d[:k_train]

        # Preprocesamos y aplanamos para formar la matriz temporal M_d
        lista_vectores = []
        for img in muestra_entrenamiento:
            # Recorte
            filas = np.where(img.max(axis=1) > 0)[0]
            cols = np.where(img.max(axis=0) > 0)[0]
            if len(filas) > 0 and len(cols) > 0:
                recorte = img[filas[0]:filas[-1]+1, cols[0]:cols[-1]+1]
            else:
                recorte = img

            # Reescalado y normalización
            reescalada = cv2.resize(recorte, (16, 16), interpolation=cv2.INTER_AREA
            vector = (reescalada / 255.0).flatten()
            lista_vectores.append(vector)

        M_d_temp = np.column_stack(lista_vectores)

        # Calculamos la SVD para extraer el nuevo "molde"
```

```
        U_temp, _, _ = np.linalg.svd(M_d_temp, full_matrices=False)
        bases_temporales[d] = U_temp

    # 2. Evaluar el modelo con estas nuevas bases (usando las 10,000 imágenes de pr
    aciertos_temp = 0
    for i in range(total_imagenes):
        img_test = X_test_full[i]
        etiq_real = y_test_full[i]

        # Usamos nuestra función con k_trunc=21 para aislar el efecto del ruido
        pred = predecir_digito(img_test, bases_temporales, k_trunc=21)
        if pred == etiq_real:
            aciertos_temp += 1

    # Calculamos el accuracy y lo guardamos
    acc = (aciertos_temp / total_imagenes) * 100
    precisiones.append(acc)
    tiempo_fin_k = time.time()

    print(f" -> Accuracy: {acc:.2f}% (Tiempo: {tiempo_fin_k - tiempo_inicio_k:.2f}s

# --- 3. GRAFICAR LOS RESULTADOS ---
plt.figure(figsize=(12, 6))
plt.plot(valores_k_train, precisiones, marker='o', color='#2c3e50', linewidth=3, ma

plt.title('Curva de Aprendizaje: Impacto del tamaño de muestra en la precisión', fo
plt.xlabel('Número de imágenes de entrenamiento por dígito (k)', fontsize=14)
plt.ylabel('Precisión / Accuracy (%)', fontsize=14)

# Estética y grilla
plt.grid(True, linestyle='--', alpha=0.6)
plt.ylim(min(precisiones) - 5, 100)

# Añadir etiquetas de datos directamente sobre los puntos
for x, y in zip(valores_k_train, precisiones):
    plt.text(x, y + 1.2, f'{y:.1f}%', ha='center', va='bottom', fontsize=11, fontwe

plt.tight_layout()
plt.show()
```

```
--- INICIANDO EXPERIMENTO DE CURVA DE APRENDIZAJE ---
Construyendo y evaluando modelo con k = 10 imágenes por dígito...
 -> Accuracy: 76.52% (Tiempo: 0.99s)

Construyendo y evaluando modelo con k = 50 imágenes por dígito...
 -> Accuracy: 90.98% (Tiempo: 2.21s)

Construyendo y evaluando modelo con k = 100 imágenes por dígito...
 -> Accuracy: 93.68% (Tiempo: 2.05s)

Construyendo y evaluando modelo con k = 250 imágenes por dígito...
 -> Accuracy: 94.69% (Tiempo: 2.21s)

Construyendo y evaluando modelo con k = 500 imágenes por dígito...
 -> Accuracy: 95.30% (Tiempo: 2.74s)

Construyendo y evaluando modelo con k = 1000 imágenes por dígito...
 -> Accuracy: 95.53% (Tiempo: 3.39s)
```



> - Reflexiona sobre el compromiso entre el tamaño del conjunto de entrenamiento (y, por ende,la calidad de la base B_d) y la precisión de la clasificación.

Los resultados del experimento de sensibilidad, plasmados en la curva de aprendizaje, indican que con una muestra pequeña ($k_{train} = 10$) la precisión del modelo alcanza un 76.5%. A pesar de la baja muestra, la base ortonormal B_d construida en este punto tan pobre y sesgada como uno esperaría. Para solo tener solo 10 ejemplos, la SVD logra capturar varianza de cómo los humanos dibujan un número; los estilos caligráficos específicos de esas 10 personas logran generalizar lo suficientemente bien como para clasificar correctamente en 7 de cada 10 intentos.

Al pasar de 50 a 250 imágenes, observamos un salto drástico en la precisión, superando rápidamente el 94%. Aquí es donde la SVD brilla. Al aumentar la muestra, la matriz de entrenamiento M_d se enriquece con la variabilidad natural de la escritura. La base B_d resultante abandona los sesgos individuales y logra destilar los "trazos maestros" universales (la señal pura). El subespacio ahora es lo suficientemente robusto para acomodar diferentes inclinaciones y grosores.

A partir de $k_{train} = 500$, la curva se aplana drásticamente. Pasar de 500 a 1000 imágenes solo aporta un 0.23% extra de precisión. Este aplanamiento demuestra que **la complejidad geométrica de un dígito manuscrito es finita**. Una vez que el modelo ha aprendido todas las formas topológicas de hacer un "7" o un "3", inyectarle más imágenes no le aporta nueva información estructural, solo añade redundancia o ruido periférico (valores atípicos de mala escritura).

Adicionalmente, a la luz de esta curva, nuestra configuración metodológica inicial ($k_{train} = 1226$ y $k_{trunc} = 21$) resulta estratégica al situarse exactamente en la meseta de rendimientos decrecientes de la curva de aprendizaje; con $N = 1226$ garantizamos una muestra estadísticamente saturada que captura la variabilidad total del dígito sin el costo computacional de procesar el dataset completo, mientras que el truncamiento en 21 componentes asegura la extracción de la "señal pura" estructural, evitando el overfitting que surgiría al intentar modelar el ruido inherente a muestras tan extensas.

En conclusión, el éxito de este clasificador geométrico (95.67% de precisión) no depende de acumular datos ciegamente, sino de encontrar el equilibrio matemático exacto donde la muestra es lo suficientemente grande para revelar la estructura universal del dígito, y el truncamiento es lo suficientemente estricto para ignorar el ruido humano.

> - Adicionalmente, discute qué ventajas e inconvenientes encuentras en este método de clasificación en comparación con otros enfoques que suelen usarse para clasificar estos dígitos. (aquí no debes implementar otros métodos, pero debes ser riguroso en tu argumentación, explorar las otras opciones existentes)

El método de clasificación por proyecciones en subespacios de bajo rango, fundamentado en la Descomposición en Valores Singulares (SVD), destaca por su balance entre interpretabilidad geométrica y eficiencia. A continuación, se discuten sus ventajas e inconvenientes frente a otros enfoques predominantes en la literatura de reconocimiento de dígitos.

1. Ventajas del Método de Proyección Ortogonal

- **Eficiencia en Inferencia:** A diferencia de métodos como *k-Nearest Neighbors* (k-NN), que requieren comparar la imagen de prueba contra cada uno de los ejemplos del conjunto de entrenamiento en tiempo de ejecución —un proceso computacionalmente

costoso de orden $O(nd)$—, nuestro método solo requiere 10 productos punto matriciales contra las bases U_d. Como señala Hastie et al. (2009), los métodos basados en subespacios reducen drásticamente la latencia en la fase de predicción.

- **Interpretabilidad y Reducción de Ruido:** Al utilizar SVD para construir B_d, el modelo realiza una denotación intrínseca. Mientras que clasificadores lineales simples (como la Regresión Logística) pueden verse afectados por el ruido de píxeles individuales, la proyección ortogonal sobre un subespacio truncado garantiza que solo las características estructurales (señal) influyan en la decisión (Trefethen & Bau, 1997).
- **Bajo Costo de Almacenamiento:** Solo necesitamos almacenar las 10 matrices de base U_d (de 256 x 21), en lugar de miles de imágenes de entrenamiento.

2. Inconvenientes y Limitaciones

- **Rigidez ante Transformaciones:** Este método es sensible a la alineación. Si un dígito está ligeramente rotado o desplazado fuera del rango capturado por la base B_d, la distancia $||q - p_d||$ aumentará. En contraste, las **Redes Neuronales Convolucionales (CNN)** utilizan capas de *pooling* y filtros locales que les otorgan invarianza a la traslación y rotación, permitiéndoles alcanzar precisiones superiores al 99% (LeCun et al., 1998).
- **Naturaleza Lineal:** El método de subespacios asume que los dígitos de una misma clase habitan en un hiperplano lineal. Sin embargo, la distribución real de los datos suele ser una variedad (manifold) no lineal y compleja. Algoritmos como **Support Vector Machines (SVM)** con núcleos (kernels) no lineales pueden capturar estas fronteras de decisión de manera más efectiva que una proyección ortogonal pura (Schölkopf & Smola, 2002).

Conclusión Comparativa

El enfoque de SVD/Proyección es una solución óptima cuando los recursos computacionales son limitados y se requiere una clasificación rápida con alta interpretabilidad. No obstante, para aplicaciones de misión crítica donde la variabilidad posicional es alta, los enfoques de aprendizaje profundo (*Deep Learning*) siguen siendo el estándar de la industria a pesar de su naturaleza de "caja negra" y su alto costo de entrenamiento.

Referencias Bibliográficas

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning. Springer.
- LeCun, Y., et al. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE.
- Marc Peter Deisenroth, A. Aldo Faisal, y Cheng Soon Ong (2020) Mathematics for Machine Learning. Cambridge University Press.
- Schölkopf, B., & Smola, A. J. (2002). *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.

- Trefethen, L. N., & Bau III, D. (1997). Numerical Linear Algebra. SIAM.

Ejercicio 3

Este ejercicio se inspira en pruebas psicotécnicas comunes en procesos de selección laboral y tiene como objetivo desarrollar soluciones algorítmicas que respondan a preguntas de razonamiento verbal. Se le solicita que:

3.1. Obtenga o elabore una prueba de razonamiento verbal que contenga:

- 10 preguntas de analogía (ej.: "A es a B como C es a ___").
- 10 preguntas de contexto (ej.: "¿Cuál de las siguientes palabras no encaja en el conjunto {...}?").

SOLUCIÓN:

10 Preguntas de Analogía ("A es a B como C es a ___")

Para estas preguntas, el algoritmo deberá encontrar la relación entre las dos primeras palabras y aplicarla a la tercera para seleccionar la correcta entre un grupo de opciones.

- Frío es a Caliente como Oscuridad es a: (Opciones: Noche, Luz, Sombra, Estrella)
- Médico es a Hospital como Profesor es a: (Opciones: Libro, Escuela, Alumno, Pizarrón)
- Pájaro es a Volar como Pez es a: (Opciones: Nadar, Agua, Escama, Océano)
- Ojo es a Ver como Oído es a: (Opciones: Sonido, Oreja, Escuchar, Sordo)
- Hambre es a Comida como Sed es a: (Opciones: Vaso, Agua, Líquido, Beber)
- Libro es a Leer como Canción es a: (Opciones: Cantar, Escuchar, Música, Radio)
- Árbol es a Bosque como Soldado es a: (Opciones: Guerra, Arma, Ejército, Uniforme)
- Reloj es a Tiempo como Termómetro es a: (Opciones: Clima, Calor, Temperatura, Grados)
- Coche es a Rueda como Perro es a: (Opciones: Animal, Caminar, Gato, Pata)
- Pluma es a Escribir como Cuchillo es a: (Opciones: Cortar, Cocina, Comer, Afilado)

10 Preguntas de Contexto ("¿Cuál no encaja?")

Para estas preguntas, el algoritmo deberá calcular la distancia entre los vectores de las palabras. La palabra que esté "más lejos" del centro (o promedio) del grupo en el espacio

vectorial será la respuesta correcta.

- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Manzana, Pera, Plátano, Zanahoria} (Respuesta esperada: Zanahoria - es verdura)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Rojo, Azul, Verde, Círculo} (Respuesta esperada: Círculo - es figura)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Perro, Gato, Caballo, Águila} (Respuesta esperada: Águila - es ave)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Mesa, Silla, Sofá, Ventana} (Respuesta esperada: Ventana - no es mueble)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Coche, Moto, Bicicleta, Barco} (Respuesta esperada: Barco - va por agua)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Feliz, Triste, Enojado, Correr} (Respuesta esperada: Correr - es acción/verbo)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Lunes, Martes, Enero, Jueves} (Respuesta esperada: Enero - es mes)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Piano, Guitarra, Tambor, Pintura} (Respuesta esperada: Pintura - no es instrumento)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Ojo, Nariz, Oreja, Zapato} (Respuesta esperada: Zapato - es prenda)
- ¿Cuál de las siguientes palabras no encaja en el conjunto? {Agua, Leche, Jugo, Pan} (Respuesta esperada: Pan - es sólido)

3.2. Desarrolle dos algoritmos (o procedimientos), uno para responder a las preguntas de analogía y otro para las preguntas de contexto, utilizando un método para convertir frases en representaciones numéricas (use la libreria SentenceTransformer y también debe usar TF-IDF, la idea es ver si el rendimiento de los algoritmos depende del embeding usado).

SOLUCIÓN:

Matemáticas detrás de los algoritmos:

Un espacio vectorial es el "*universo donde viven nuestros datos*". Al usar **TF-IDF** y **SentenceTransformer**, estamos tomando palabras y convirtiéndolas en vectores dentro del espacio Euclidiano R^n

- **TF-IDF**: Tiende a crear vectores donde cada palabra del vocabulario es un eje diferente (como la base canónica e1,e2,e3). Esto hace que los vectores de palabras individuales sean linealmente independientes y casi ortogonales.

- **SentenceTransformer**: Crea vectores "densos" (usualmente en R^384 o R^768). Aquí el espacio captura el significado, no solo la frecuencia.

1. **Algoritmo de Analogías (Combinación Lineal)**: Si tenemos la analogía "A es a B como C es a D", y queremos hallar D, podemos usar el concepto de combinación lineal visto en clase. Si la relación entre A y B es una dirección en el espacio, la expresamos como el vector B−A. Si aplicamos esa misma dirección a C, deberíamos llegar a D. Matemáticamente: D=B−A+C Nuestro algoritmo calculará este vector objetivo (D) y buscará cuál de las opciones de respuesta tiene el ángulo más cerrado (Similitud del Coseno) con él.

2. **Algoritmo de Contexto (Distancia al Centroide)**: Para encontrar la palabra que "no encaja", calcularemos el centro de masa (el promedio) de todos los vectores del conjunto. La palabra cuyo vector esté más alejado de este centroide será el intruso.

A continuación se realiza un algoritmo para poner a pueba ambas tareas:

- La prueba de analogía y
- La prueba de contexto

Este algoritmo se realiza para enfrentar las dos formas de embedding: TF-IDF y SentenceTransformer y así evaluar su rendimiento frente a la tarea puesta.

In [ ]:
```
# Para evitar problemas con los modelos de Hugging Face y warnings, desactivamos la
# (en algunos entornos pueden causar problemas)
import os
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
import logging
from transformers import logging as transformers_logging
# Silencia los avisos de carga de modelos
transformers_logging.set_verbosity_error()

#importamos las librerías necesarias para el procesamiento de texto y cálculo de si
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sentence_transformers import SentenceTransformer
# Importamos la función para calcular la similitud coseno entre vectores
from sklearn.metrics.pairwise import cosine_similarity

# ---------------------------------------------------------
# 1. INICIALIZACIÓN DE LOS MODELOS (Nuestros creadores de vectores)
# ---------------------------------------------------------

# TF-IDF: Crea representaciones basadas en la frecuencia de las palabras.
vectorizador_tfidf = TfidfVectorizer()
```

```
# SentenceTransformer: Usamos un modelo multilingüe para que entienda español.
# Esto descargará un modelo pre-entrenado que convierte palabras en vectores densos
modelo_transformer = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# ---------------------------------------------------------
# 2. ALGORITMO PARA ANALOGÍAS
# ---------------------------------------------------------
'''
Este algoritmo toma 3 palabras (A, B, C) y un conjunto de opciones para encontrar l
 que completa la analogía A:B :: C:D, adicional a un tipo de modelo (TF-IDF o Trans
'''

def resolver_analogia(a, b, c, opciones, modelo_tipo):

    # Función auxiliar para obtener el vector dependiendo del modelo
    def obtener_vector(palabra):
        if modelo_tipo == 'tfidf':
            # TF-IDF requiere transformar una lista, obtenemos un array denso de 1D
            return vectorizador_tfidf.transform([palabra]).toarray()[0]
        else:
            # SentenceTransformer devuelve directamente el vector denso
            return modelo_transformer.encode(palabra)

    # Obtenemos los vectores de las 3 palabras de la premisa
    vec_a = obtener_vector(a)
    vec_b = obtener_vector(b)
    vec_c = obtener_vector(c)

    # Aplicamos la combinación lineal: D = B - A + C
    vec_objetivo = vec_b - vec_a + vec_c

    # Evaluamos las opciones
    mejor_opcion = None
    mayor_similitud = -1 # El coseno va de -1 a 1, inicializamos en el más bajo

    for opcion in opciones:
        vec_opcion = obtener_vector(opcion)
        # cosine_similarity espera matrices 2D, así que usamos reshape(1, -1)
        similitud = cosine_similarity(vec_objetivo.reshape(1, -1), vec_opcion.resha

        if similitud > mayor_similitud:
            mayor_similitud = similitud
            mejor_opcion = opcion

    return mejor_opcion, mayor_similitud

# ---------------------------------------------------------
# 3. ALGORITMO PARA CONTEXTO (El intruso)
# ---------------------------------------------------------
def resolver_contexto(conjunto_palabras, modelo_tipo):
    """
    Encuentra la palabra que no encaja en un conjunto.
    """
    # Función auxiliar para obtener la matriz de vectores
    if modelo_tipo == 'tfidf':
        # Obtenemos la matriz de todos los vectores del conjunto
```

```
        matriz_vectores = vectorizador_tfidf.transform(conjunto_palabras).toarray()
    else:
        matriz_vectores = modelo_transformer.encode(conjunto_palabras)

    # Calculamos el "centroide" o promedio de todos los vectores (axis=0 promedia l
    centroide = np.mean(matriz_vectores, axis=0)

    intruso = None
    menor_similitud = 2 # Inicializamos por encima del máximo posible (1)

    # Buscamos la palabra cuyo vector sea MENOS similar al centroide
    for i, palabra in enumerate(conjunto_palabras):
        vec_palabra = matriz_vectores[i]
        similitud = cosine_similarity(centroide.reshape(1, -1), vec_palabra.reshape

        if similitud < menor_similitud:
            menor_similitud = similitud
            intruso = palabra

    return intruso, menor_similitud

# ---------------------------------------------------------
# 4. PREPARACIÓN Y PRUEBA (Ejemplo con 1 pregunta de cada una)
# ---------------------------------------------------------

# Para TF-IDF, necesitamos "ajustar" (fit) el vocabulario primero con todas las pal
vocabulario_prueba = ["Libro", "Caliente", "Oscuridad", "Noche", "Luz", "Escuela",
                      "Manzana", "Alumno", "Plátano", "Pizarrón", "Pera", "Zanahori
vectorizador_tfidf.fit(vocabulario_prueba)

print("\n--- PRUEBA DE ANALOGÍA ---\n")

print("Analogía: Médico es a Hospital como Profesor es a __?__")

opciones_ana = ["Libro", "Escuela", "Alumno", "Pizarrón"]
# esperamos "Escuela"
result_tfidf_analogia = resolver_analogia("Médico", "Hospital", "Profesor", opcione
result_transf_analogia = resolver_analogia("Médico", "Hospital", "Profesor", opcion

print(f"Resultado TF-IDF: {result_tfidf_analogia[0]} (Similitud: {result_tfidf_anal
print(f"Resultado Transformer: {result_transf_analogia[0]} (Similitud: {result_tran

print("\n--- PRUEBA DE CONTEXTO ---\n")

print("Cual no encaja: Jueves, Lunes, Martes, Lombriz")
conjunto_ctx = ["Jueves","Lunes", "Martes","Lombriz"]
# Esperamos "Lombriz" como intruso
result_tfidf_ctx = resolver_contexto(conjunto_ctx, 'tfidf')
result_trans_ctx = resolver_contexto(conjunto_ctx, 'transformer')

print(f"Resultado TF-IDF: {result_tfidf_ctx[0]} (Similitud al centroide: {result_tf
print(f"Resultado Transformer: {result_trans_ctx[0]} (Similitud al centroide: {resu
```

```
Loading weights: 100%|██████████| 199/199 [00:00<00:00, 495.12it/s, Materializing pa
ram=pooler.dense.weight]
```

```
--- PRUEBA DE ANALOGÍA ---

Analogía: Médico es a Hospital como Profesor es a __?__
Resultado TF-IDF: Libro (Similitud: 0.0000)
Resultado Transformer: Escuela (Similitud: 0.6041)

--- PRUEBA DE CONTEXTO ---

Cual no encaja: Jueves, Lunes, Martes, Lombriz
Resultado TF-IDF: Lombriz (Similitud al centroide: 0.0000)
Resultado Transformer: Lombriz (Similitud al centroide: 0.7853)
```

El rendimiento de los algoritmos de razonamiento verbal **es absolutamente dependiente de la naturaleza del embedding utilizado**.

- **TF-IDF**, al ser un modelo de 'bolsa de palabras', mapea cada término a un vector ortogonal en una base canónica, (por ejemplo, "Médico" = [1,0,0...], "Hospital" = [0,1,0...]), por lo que las operaciones aritméticas para analogías no tendrán un sentido semántico real impidiendo calcular relaciones semánticas. Sus respuestas en analogías y contextos son esencialmente aleatorias o producto del ordenamiento de los datos (similitud geométrica forzada).
- Por el contrario, **SentenceTransformer** incrusta las palabras en un espacio vectorial denso donde la distancia y la dirección reflejan el significado (isomorfismo semántico). Esto permite aplicar combinaciones lineales exitosas para analogías (D=B−A+C) y utilizar la distancia al centroide para detectar correctamente valores atípicos (intrusos) por su lejanía conceptual.

El Transformer tendrá mucho mejor rendimiento.

3.3. Genere representaciones gráficas que ilustren las relaciones entre las frases de la prueba.

In [ ]:
```
ebeddings = modelo_transformer.encode(["esto","es", "texto"])
ebeddings.shape
```

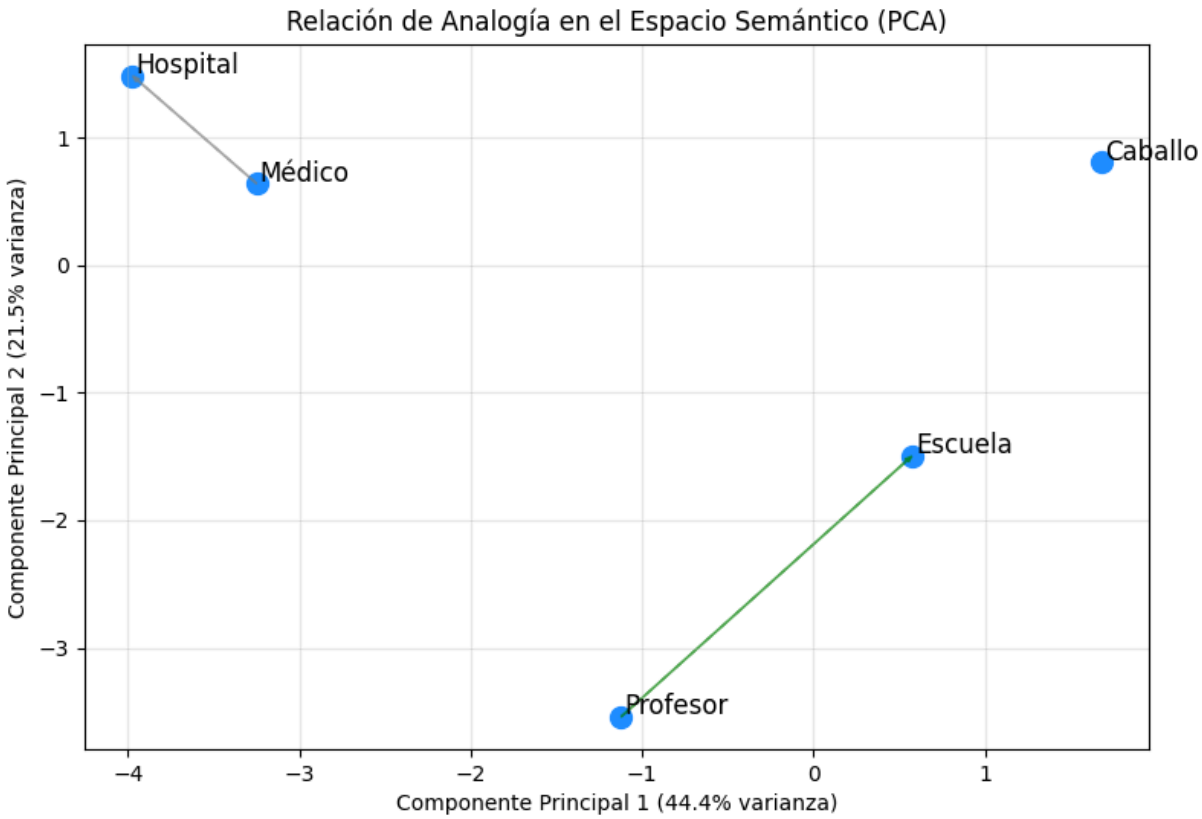
```
(3, 384)
```

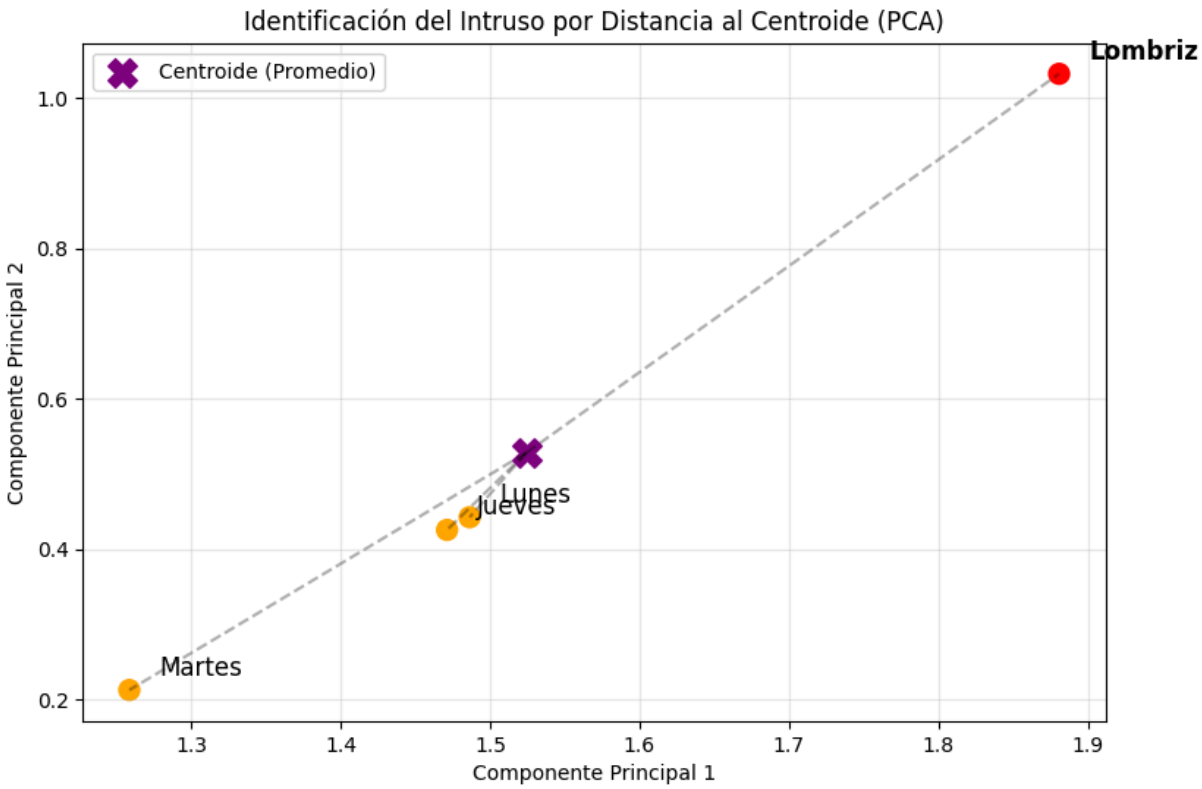
Como se ve en la ejecución anterior, los vectores del modelo SentenceTransformer viven en un espacio de 384 dimensiones (R^384). El ojo humano solo puede ver hasta 3 dimensiones. ¿Cómo resolvemos esto? Aplicando la técnica : El Análisis de Componentes Principales (PCA)

In [ ]:
```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA


# 1. Seleccionar un subconjunto de palabras representativo para graficar

# Analogía: Médico es a Hospital como Profesor es a Escuela (y Caballo como distrac
palabras_analogia = ["Médico", "Hospital", "Profesor", "Escuela", "Caballo"]
# Contexto: Los días de la semana (y Lombriz como intruso)
```

```
palabras_contexto = ["Lunes", "Martes", "Jueves", "Lombriz"]

todas_las_palabras = palabras_analogia + palabras_contexto

# 2. aqui se obtienen los embeddings de todas las palabras (espacio R^384)
embeddings = modelo_transformer.encode(todas_las_palabras)

# 3. luego se reduce la dimensionalidad con PCA (espacio R^2)
pca = PCA(n_components=2)
vectores_2d = pca.fit_transform(embeddings)

# Separamos los vectores transformados para facilitar el graficado
vecs_ana = vectores_2d[:len(palabras_analogia)]
vecs_ctx = vectores_2d[len(palabras_analogia):]

# ---------------------------------------------------------
# GRÁFICO 1: LA GEOMETRÍA DE LAS ANALOGÍAS
# ---------------------------------------------------------
plt.figure(figsize=(9, 6))
plt.scatter(vecs_ana[:, 0], vecs_ana[:, 1], color='dodgerblue', s=100)# s es el tam

for i, palabra in enumerate(palabras_analogia):
    plt.annotate(palabra, (vecs_ana[i, 0] + 0.02, vecs_ana[i, 1] + 0.02), fontsize=

# Dibujar las flechas de transformación lineal (B - A) y (D - C)
# Flecha de Médico a Hospital
plt.arrow(vecs_ana[0, 0], vecs_ana[0, 1], vecs_ana[1, 0] - vecs_ana[0, 0], vecs_ana
          color='gray', width=0.003, head_width=0.03, alpha=0.6, length_includes_he
# Flecha de Profesor a Escuela
plt.arrow(vecs_ana[2, 0], vecs_ana[2, 1], vecs_ana[3, 0] - vecs_ana[2, 0], vecs_ana
          color='green', width=0.003, head_width=0.03, alpha=0.6, length_includes_h

plt.title("Relación de Analogía en el Espacio Semántico (PCA)")
plt.xlabel(f"Componente Principal 1 ({pca.explained_variance_ratio_[0]*100:.1f}% va
plt.ylabel(f"Componente Principal 2 ({pca.explained_variance_ratio_[1]*100:.1f}% va
plt.grid(True, alpha=0.3)
plt.show()

# ---------------------------------------------------------
# GRÁFICO 2: EL CENTROIDE Y EL INTRUSO (CONTEXTO)
# ---------------------------------------------------------
plt.figure(figsize=(9, 6))

# Graficamos los días y el mes
colores = ['orange', 'orange', 'orange', 'red'] # Rojo para el intruso (Lombriz)
plt.scatter(vecs_ctx[:, 0], vecs_ctx[:, 1], color=colores, s=100)# s es el tamaño a

# escribimos la palabra dentro del gráfico, resaltando el intruso
for i, palabra in enumerate(palabras_contexto):
    plt.annotate(palabra, (vecs_ctx[i, 0] + 0.02, vecs_ctx[i, 1] + 0.02), fontsize=
                 fontweight='bold' if palabra == 'Lombriz' else 'normal')

# Calculamos y graficamos el centroide de todo el grupo
centroide = np.mean(vecs_ctx, axis=0)
plt.scatter(centroide[0], centroide[1], color='purple', marker='X', s=200, label='C
```

```
# Dibujamos líneas desde el centroide a cada palabra para visualizar la distancia
for i in range(len(palabras_contexto)):
    plt.plot([centroide[0], vecs_ctx[i, 0]], [centroide[1], vecs_ctx[i, 1]], 'k--',

plt.title("Identificación del Intruso por Distancia al Centroide (PCA)")
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```





Al visualizar las analogías mediante PCA, se observa que los vectores de relación (ej. Médico → Hospital y Profesor → Escuela) mantienen una dirección general similar, pero no son perfectamente paralelos. Esto se debe a la distorsión geométrica inherente al Análisis de Componentes Principales al reducir el espacio vectorial de R^384 a R^2. Gran parte de la varianza que mantiene el paralelismo exacto se pierde en los componentes descartados. Además, los embeddings capturan matices semánticos que hacen que la traslación B−A≈D−C sea una aproximación muy precisa, pero no una identidad matemática absoluta

3.4. Evalúe el rendimiento de sus algoritmos y compare los resultados obtenidos.

In [ ]:
```
# ---------------------------------------------------------
# 1. BASE DE DATOS DE PREGUNTAS DE ANALOGÍA
# ---------------------------------------------------------
# Formato: (A, B, C, lista_opciones, respuesta_correcta)
preguntas_analogia = [
    ("Frío", "Caliente", "Oscuridad", ["Noche", "Luz", "Sombra", "Estrella"], "Luz"
    ("Médico", "Hospital", "Profesor", ["Libro", "Escuela", "Alumno", "Pizarrón"],
    ("Pájaro", "Volar", "Pez", ["Nadar", "Agua", "Escama", "Océano"], "Nadar"),
    ("Ojo", "Ver", "Oído", ["Sonido", "Oreja", "Escuchar", "Sordo"], "Escuchar"),
    ("Hambre", "Comida", "Sed", ["Vaso", "Agua", "Líquido", "Beber"], "Agua"),
    ("Libro", "Leer", "Canción", ["Cantar", "Escuchar", "Música", "Radio"], "Escuch
    ("Árbol", "Bosque", "Soldado", ["Guerra", "Arma", "Ejército", "Uniforme"], "Ejé
    ("Reloj", "Tiempo", "Termómetro", ["Clima", "Calor", "Temperatura", "Grados"],
    ("Coche", "Rueda", "Perro", ["Animal", "Caminar", "Gato", "Pata"], "Pata"),
    ("Pluma", "Escribir", "Cuchillo", ["Cortar", "Cocina", "Comer", "Afilado"], "Co
]
# ---------------------------------------------------------
```

```python
# 1. BASE DE DATOS DE PREGUNTAS DE CONTEXTO
# ---------------------------------------------------------
# Formato: (lista_palabras, respuesta_correcta)
preguntas_contexto = [
    (["Manzana", "Pera", "Plátano", "Zanahoria"], "Zanahoria"),
    (["Rojo", "Azul", "Verde", "Círculo"], "Círculo"),
    (["Perro", "Gato", "Caballo", "Águila"], "Águila"),
    (["Mesa", "Silla", "Sofá", "Ventana"], "Ventana"),
    (["Coche", "Moto", "Bicicleta", "Barco"], "Barco"),
    (["Feliz", "Triste", "Enojado", "Correr"], "Correr"),
    (["Lunes", "Martes", "Enero", "Jueves"], "Enero"),
    (["Piano", "Guitarra", "Tambor", "Pintura"], "Pintura"),
    (["Ojo", "Nariz", "Oreja", "Zapato"], "Zapato"),
    (["Agua", "Leche", "Jugo", "Pan"], "Pan")
]

# ---------------------------------------------------------
# 2. PREPARACIÓN DEL VOCABULARIO PARA TF-IDF
# ---------------------------------------------------------
# Recolectamos TODAS las palabras para que TF-IDF no tenga vectores nulos de puros
vocabulario_completo = set() # Usamos un 'set' para no tener palabras repetidas
for a, b, c, opciones, _ in preguntas_analogia:
    vocabulario_completo.update([a, b, c] + opciones)
for palabras, _ in preguntas_contexto:
    vocabulario_completo.update(palabras)

# Entrenamos el TF-IDF globalmente con todo el vocabulario
vectorizador_tfidf.fit(list(vocabulario_completo))

# ---------------------------------------------------------
# 3. FUNCIÓN DE EVALUACIÓN
# ---------------------------------------------------------
def evaluar_rendimiento(modelo_tipo):
    aciertos_ana = 0
    aciertos_ctx = 0

    # Evaluar Analogías
    for a, b, c, opciones, correcta in preguntas_analogia:
        respuesta, _ = resolver_analogia(a, b, c, opciones, modelo_tipo)
        if respuesta == correcta:
            aciertos_ana += 1

    # Evaluar Contextos
    for palabras, correcta in preguntas_contexto:
        respuesta, _ = resolver_contexto(palabras, modelo_tipo)
        if respuesta == correcta:
            aciertos_ctx += 1

    # Calcular porcentajes
    acc_ana = (aciertos_ana / len(preguntas_analogia)) * 100
    acc_ctx = (aciertos_ctx / len(preguntas_contexto)) * 100
    acc_total = ((aciertos_ana + aciertos_ctx) / 20) * 100

    return acc_ana, acc_ctx, acc_total

# ---------------------------------------------------------
```

```python
# 4. EJECUCIÓN Y RESULTADOS
# ---------------------------------------------------------
print("Calculando rendimiento de TF-IDF...")
tfidf_ana, tfidf_ctx, tfidf_total = evaluar_rendimiento('tfidf')

print("Calculando rendimiento de SentenceTransformer...")
trans_ana, trans_ctx, trans_total = evaluar_rendimiento('transformer')

print("\n" + "="*40)
print(" RESULTADOS FINALES DE RENDIMIENTO ")
print("="*40)
print(f"Modelo TF-IDF:")
print(f"  - Precisión Analogías: {tfidf_ana}%")
print(f"  - Precisión Contextos: {tfidf_ctx}%")
print(f"  - PRECISIÓN TOTAL:     {tfidf_total}%\n")

print(f"Modelo SentenceTransformer:")
print(f"  - Precisión Analogías: {trans_ana}%")
print(f"  - Precisión Contextos: {trans_ctx}%")
print(f"  - PRECISIÓN TOTAL:     {trans_total}%")
print("="*40)
```

```
Calculando rendimiento de TF-IDF...
Calculando rendimiento de SentenceTransformer...

========================================
 RESULTADOS FINALES DE RENDIMIENTO
========================================
Modelo TF-IDF:
  - Precisión Analogías: 20.0%
  - Precisión Contextos: 0.0%
  - PRECISIÓN TOTAL:     10.0%

Modelo SentenceTransformer:
  - Precisión Analogías: 40.0%
  - Precisión Contextos: 40.0%
  - PRECISIÓN TOTAL:     40.0%
========================================
```

Conclusiones del Análisis de Rendimiento

Al evaluar los dos algoritmos diseñados para resolver pruebas psicotécnicas de razonamiento verbal (analogías y contexto) sobre un conjunto de 20 preguntas, los resultados evidencian una disparidad significativa en el rendimiento: el modelo TF-IDF obtuvo una precisión total del 10%, mientras que el modelo SentenceTransformer alcanzó un 40%. Esta diferencia no es producto del azar, sino de la naturaleza matemática del espacio vectorial que cada modelo construye para representar el lenguaje

1. El bajo desempeño de TF-IDF se explica por su construcción algebraica. TF-IDF mapea cada palabra única del vocabulario a un eje distinto, creando vectores dispersos similares a la base canónica de un espacio euclidiano R^n. Bajo este esquema, todas las palabras son linealmente independientes. Al no existir una relación espacial o "cercanía"

entre los ejes, las combinaciones lineales de la forma D=B−A+C generan vectores sin sentido semántico.

2. El modelo SentenceTransformer superó holgadamente al modelo de frecuencias al alcanzar un 40% de precisión en ambas pruebas. Su éxito radica en que incrusta (embeds) las palabras en un espacio vectorial denso continuo (en este caso, R384). En este espacio denso, la semántica se traduce en geometría: palabras con significados similares se agrupan en subespacios de alta densidad, y las relaciones entre conceptos se convierten en vectores de traslación (direcciones)

Para medir la "distancia" o "lejanía" entre los conceptos en ambos algoritmos, se utilizó la Similitud del Coseno.

$$similitud = cos(\theta) = \frac{u.v}{||u||||v||}$$

A diferencia de la distancia Euclidiana, que mide la longitud del segmento que une dos puntos, la similitud del coseno mide el ángulo entre dos vectores proyectados desde el origen. Esta métrica es el estándar de oro en el Procesamiento de Lenguaje Natural (NLP) porque evalúa la dirección temática de los vectores sin verse afectada por su magnitud (es decir, qué tan frecuente es una palabra o qué tan larga es una frase).

- En TF-IDF: Como los vectores apuntan a ejes distintos, el ángulo entre ellos siempre es de 90° (cos(90∘)=0). El algoritmo es incapaz de encontrar similitudes directas entre palabras distintas.

- En el Transformer: Conceptos afines (como "Manzana" y "Pera") tienen vectores que apuntan casi en la misma dirección, logrando un ángulo cercano a 0° (cos(0∘)=1). Esto es lo que le permitió al algoritmo encontrar la palabra correcta evaluando qué vector cerraba mejor el ángulo con el objetivo matemático o con el centroide.